

F24-106-D-CodeNexus

Project Team

Malaika Zafar	21I-1110
Talal Habib Malik	21I-1111
Amna Shehzad	21I-1209

Session 2021-2025

Supervised by

Dr. Atif Jilani



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

October, 2024

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	1
1.2.0.1	Bloaters	2
1.2.0.2	Object Oriented Abusers	2
1.2.0.3	Dispensables	2
1.2.0.4	Couplers	2
1.2.0.5	Others	3
1.3	Modules	3
1.3.1	Module 1 - Codebase Analysis	3
1.3.2	Module 2 - Detection Module	3
1.3.3	Module 3 - Refactoring Module	4
1.3.4	Module 4 - Change logs Module	5
1.3.5	Module 5 - Rulesets Module	5
1.3.6	Module 6 - Website development Module	5
1.4	User Classes and Characteristics	6
2	Project Requirements	9
2.1	Use-case	9
2.1.1	UC-01: Identify Code Smells in Real-Time	11
2.1.2	UC-02: Modify Project	12
2.1.3	UC-03: Trigger Code Smell Detection	13
2.1.4	UC-04: Check Refactor Readiness	14
2.1.5	UC-05: Perform automated refactoring	15
2.1.6	UC-06: Filter Code Smells	16
2.1.7	UC-07: Revert Code Changes	17
2.1.8	UC-08: Configure Custom Rulesets	18
2.1.9	UC-09: View Codebase Logs	19
2.1.10	UC-10: Share Codebase Logs	20
2.1.11	UC-11: Isolated Compilation and Testing of Refactored Code	21
2.1.12	UC-12: Visualizing Inter-File Dependencies	22
2.2	Functional Requirements	22

2.2.1	Module 1 - Codebase Analysis	22
2.2.2	Module 2 - Detection Module	23
2.2.3	Module 3 - Refactoring Module	25
2.2.4	Module 4 - Change Logs Module	26
2.2.5	Module 5 - Rulesets Module	27
2.2.6	Module 6 - Website Development Module	27
2.3	Non-Functional Requirements	28
2.3.1	Reliability	28
2.3.2	Usability	29
2.3.3	Performance	29
2.3.4	Security	29
2.3.5	Scalability	29
3	System Overview	31
3.1	Architectural Design	31
3.2	Design Models	32
3.3	Data Design	56
	References	63

List of Figures

2.1	Use Case For CodeNexus	10
3.1	Architecture of CodeNexus	32
3.2	UC-01 Activity Diagram	33
3.3	UC-02 Activity Diagram	34
3.4	UC-03 Activity Diagram	35
3.5	UC-04 Activity Diagram	36
3.6	UC-05 Activity Diagram	37
3.7	UC-06 Activity Diagram	38
3.8	UC-07 Activity Diagram	39
3.9	UC-08 Activity Diagram	40
3.10	UC-09 Activity Diagram	41
3.11	UC-10 Activity Diagram	42
3.12	UC-11 Activity Diagram	43
3.13	UC-12 Activity Diagram	44
3.14	Data Flow Diagram	45
3.15	UC-01: System Sequence	46
3.16	UC-02: System Sequence	47
3.17	UC-03 System Sequence	48
3.18	UC-05 System Sequence	49
3.19	UC-06 System Sequence	50
3.20	UC-07 System Sequence	51
3.21	UC-08 System Sequence	52
3.22	UC-09 System Sequence	53
3.23	UC-10 System Sequence	54
3.24	UC-11 System Sequence	55
3.25	UC-12 System Sequence	56
3.26	JSON Schema for CodeNexus	59

List of Tables

2.1	UC-01	11
2.2	UC-02	12
2.3	UC-03	13
2.4	UC-04	14
2.5	UC-05	15
2.6	UC-06	16
2.7	UC-07	17
2.8	UC-08	18
2.9	UC-09	19
2.10	UC-10	20
2.11	UC-11	21
2.12	UC-12	22

Chapter 1

Introduction

The purpose of this scope document is to outline the development of a VS Code extension that automatically identifies and refactors code smells across entire Python projects. This the document outlines the project's purpose, problem statement, and general objectives. It also clarifies the project's technical approach, scope, and possible influence on enhancing code quality and maintainability.

1.1 Problem Statement

According to the 2024 developer survey conducted by Stack Overflow, about 51% [2] of the developers work with Python [3] and 35% of these developers use VS Code [2] [1]. The existing tools for the detection of code smells operate on a single-file basis and do not consider the inter-file dependencies and other project-wide issues that contribute to code smells. Therefore, we believe that there exists the need for a better tool for the detection of code smells across multiple files and aiding in refactoring of such problematic code.

1.2 Scope

In this project, we aim to create a Visual Studio Code extension that can identify and refactor a range of code smells. Our main objective in developing this tool is to help in creating clean, efficient and maintainable code bases. The tool will only target the python files available in the open folder. The tool will be able to identify up to 21 code smells which are listed below:

1.2.0.1 Bloaters

- Long Functions
- Long Parameter List
- Too Many Nested Loops
- Overly Complex Conditional Statements
- God Object

1.2.0.2 Object Oriented Abusers

- Switch Statement Abuser
- Temporary Field
- Over-Encapsulation (Optional)

1.2.0.3 Dispensables

- Dead Code
- Duplicated Code
- Unreachable Code
- Duplicated Logic (Optional)
- Comments as Code (Optional)

1.2.0.4 Couplers

- Feature Envy
- Middle Man
- Inappropriate Intimacy

1.2.0.5 Others

- Unutilized Variables
- Magic Numbers
- Variables Naming Notation
- Global Variables Conflict
- Excessive Use of Flags
- Non-Descriptive Method Names (Optional)
- Inconsistent Naming (Optional)

1.3 Modules

1.3.1 Module 1 - Codebase Analysis

1. **Project Structure Extraction:** The tool will go through and analyse the currently active project to extract the folder structure which will help in extracting information. The system will identify all the relevant files (.py files) as well as extract and structure the content within them.
2. **Abstract Syntax Tree Generation:** Using the previously structured code, the tool will generate an Abstract Syntax Tree (AST) for each file. The ASTs will provide a representation of the code, allowing us to perform a in-depth analysis of the code.
3. **Code Dependency Graph Generation:** The tool analyze the imports and create a code dependency graph. This graph will show how different files are related to each other, facilitating the analysis of code smells and visualizing the inter-file dependencies that exist within the project.

1.3.2 Module 2 - Detection Module

1. **Real Time Detection of Code Smells:** The tool, upon opening a project, starts a real-time detection process for code smells. As users make changes to the code and it becomes compilable, AST analysis is performed on the modified file.
2. **User Triggered Detection of Code Smells:** In the tool, the detection of certain subset of code smells will need to be manually triggered by the user. By default, it will

only detect code smells present in the currently opened file. However, if needed, the user can choose to scan the entire project to find these issues everywhere.

1.3.3 Module 3 - Refactoring Module

1. **Refactoring Readiness Assessment of Identified Code Smells:** The system will evaluate whether identified code smells are suitable for automated refactoring by large language models (LLMs). This feature will include the analysis of various factors, including code complexity, and dependency relationships, to determine if an LLM can effectively refactor the code.
2. **Refactoring of Identified Code Smells:** After detection of code smell, users can then choose to either manually adjust the code or utilize the automatic refactoring option to change their code depending on the results of the Refactoring Readiness analysis.
3. **Filter and Refactor specific code based on detected code smells:** By filtering code smells such as long functions, inconsistent naming, and god objects, users can decide which issues they would like to address first and apply specific refactoring techniques to improve code quality.
4. **Cascading Code Refactoring:** The refactoring process ensures that every impacted file within the code base is consistently updated. Any modifications made to one file will be reflected across the entire code base.
5. **Isolated Compilation and Testing of Refactored Code base:** Users can compile and run a temporary version of the refactored code base before accepting the changes, allowing them to validate and test the modifications without altering the original code base.
6. **Isolated Compilation and Testing of Refactored Code (Supplementary):** Users can compile and run a temporary version of the refactored code before accepting the changes, allowing them to validate and test whether the modifications are semantically same.
7. **Personalized Refactoring Suggestions (Supplementary):** Refactoring suggestions will be tailored to the user's coding style and patterns. When a user writes code, the tool analyzes the user's coding practices. It then generates refactoring recommendations that align with these preferences. This personalized approach ensures that the refactored code remains consistent with the user's coding practices.

1.3.4 Module 4 - Change logs Module

1. **Code smell detection and Refactoring tracker:** This tracks the detection and refactoring of code smells, while also keeping track the structure and dependencies between files. It provides a detailed history of how code smells have been managed.
2. **File Dependency Tracker:** It also tracks the inter-dependencies within the project that have evolved over time, offering developers a clear understanding of the project's progression and the impact of their changes.
3. **Revert Readiness Assessment:** This feature is to ensure that any refactoring that the user wants to revert back to his original code is analyzed and determined to be possible to be reverted without causing error in the code or inconsistency.
4. **Code rollback:** This feature allows users to rollback certain changes made through automated refactoring given that the code snippet has passed the revert readiness assessment.

1.3.5 Module 5 - Rulesets Module

1. **VS Code Extension Rulesets:**
 - (a) **Rulesets for Detection:** Users can specify certain rules that the tool will consider during code smell detection. For example:
 - i. Include and Exclude Detection of Certain Code Smells for a Repository
 - ii. Include and Exclude Files for Detection of Code Smells
 - (b) **Rulesets for Refactoring:** Users can specify certain rules that the tool will consider during automated code refactoring. For example:
 - i. Include and Exclude Refactoring of Certain Code Smells for a Repository
 - ii. Include and Exclude Files for Refactoring of Code Smells

1.3.6 Module 6 - Website development Module

1. **Shareable Logs Access:** The user can add viewers to their project. These viewers will be able to view and analyze the history and evolution of different code smells over the life cycle of the project. This ensures that the viewers fully grasp the evolution of code smells within a code base over time and how they were handled or refactored.

2. **Project Visualization:** This allows users to view the entire project structure, including the dependencies between code files. This interactive visualization will enable the users to view and analyze the project. This will aid their understanding of how different components of the project are connected.
3. **Inter-File Impact Analysis:** The user would be able to view and understand how modifications in one file would impact other files present in the code base. This helps them understand the impact of the changes and keep track of how different files are connected with each other.
4. **Code Smells Timeline:** The tool will maintain a timeline of identified code smells, along with specific details about them. It will also track refactored code, allowing users to view the changes made over time.

1.4 User Classes and Characteristics

User class	Description
Software Developers	Role: Actively interact with the VS Code extension for code smell detection, refactoring readiness assessment, and automated refactoring. Reason: They are the main users who will benefit directly from the tool in their daily development workflow.
Quality Assurance (QA) Engineers	Role: Use the extension to ensure that the code is clean and meets quality standards. They may run code smell detection and assist in verifying that refactoring aligns with coding standards. Reason: QA engineers are directly involved in maintaining code quality and will use the tool to identify and fix code smells.
Academics (Computer related students)	Role: Use the extension as a learning tool for better understanding code quality, identifying code smells, and practicing refactoring. Reason: They are secondary actors since their primary interaction with the tool is for educational purposes rather than professional development.
Open-Source Contributors	Role: Contribute to the development of the extension or the projects where the extension is used. They may use the tool for collaborative code review, refactoring, and improvements. Reason: While they actively use the tool, their interaction is less frequent and more collaborative, compared to the primary actors who use it as part of their daily development tasks.
Project Managers or Team Leads	Role: Oversee code quality and consistency across the team's code base. They may use the tool to analyze code smell history, review refactoring readiness, and ensure that coding standards are followed. Reason: They influence the use of the tool and its adoption in projects but are not directly engaged in day-to-day operations.
Maintainers of the VS Code Extension (Extension Developers)	Role: Develop, maintain, and update the extension itself, ensuring it meets user needs and addresses detected bugs. Reason: They influence how the extension functions but do not use it for development or code quality purposes.
DevOps Engineers or System Administrators	Role: Might occasionally use the extension for code analysis related to deployment scripts or configuration files in Python, ensuring those areas are clean and maintainable. Reason: Their interaction is limited and incidental, as their primary focus is on deployment and system stability, not code refactoring.

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of our project.

2.1 Use-case

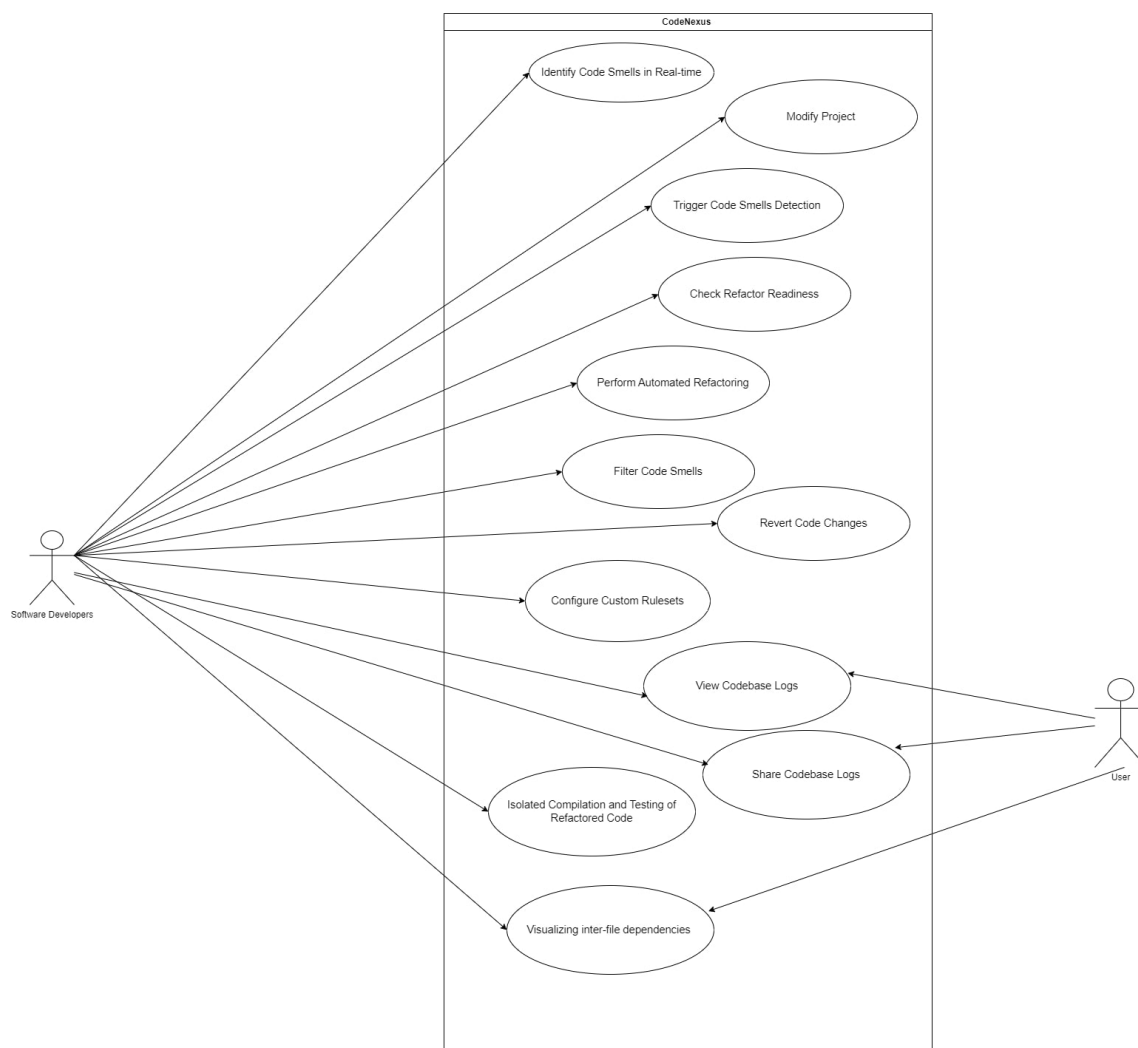


Figure 2.1: Use Case For CodeNexus

2.1.1 UC-01: Identify Code Smells in Real-Time

Use Case Name	Identify code smells in real time
Scope	CodeNexus
Level	User Goals
Primary actor	Developers
Stakeholders and Interests	Developers: Developers want to ensure that the code he is writing remains maintainable and understandable.
Pre-Conditions	1. CodeNexus extension must be installed on the developer's Visual Studio Code 2. CodeNexus extension must be activated on the developer's Visual Studio Code.
Post-Conditions	Successful detection of the code smells in code, if any are present
Main Success Scenario	1. The use case starts when the developer opens a python project. 2. The system extracts content of all the .py files. 3. The system then forwards the extracted content to the microservice for real-time detection of certain code smell. 4. The system receives detection data for all the files from the microservice. 5. The system performs final inter-file code smells detection tasks. 6. The system displays the results received from the microservice.
Extensions	2. A python file is not compilable. 2.1. The system does not detect any code smells for that file.

Table 2.1: UC-01

2.1.2 UC-02: Modify Project

Use Case Name	Modify project
Scope	CodeNexus
Level	User Goals
Primary actor	Developers
Stakeholders and Interests	Developer: Developer wants to write python code while ensuring that it remains readable, maintainable and understandable.
Pre-Conditions	1. CodeNexus extension must be installed on the developer's Visual Studio Code 2. CodeNexus extension must be activated on the developer's Visual Studio Code. 3. A python project is open in the vscode workspace.
Post-Conditions	Successful detection of the code smells in code if any are present
Main Success Scenario	1. The use case starts when the developer makes a change in the project folder or one of the project files. 2. The system extracts content of the modified .py file. 3. The system then forwards the extracted content to the micro-service. 4. The system runs real-time code smell detection in the updated project. 5. The system receives detection data from the microservice. 6. The system performs final inter-file code smells detection tasks. 7. The system displays the result received from the microservice.
Extensions	2. A python file is not compilable. 2.1. The system does not detect any code smells for that file.

Table 2.2: UC-02

2.1.3 UC-03: Trigger Code Smell Detection

Use Case Name	Trigger code smell detection
Scope	CodeNexus
Level	User Goals
Primary actor	Developers
Stakeholders and Interests	Developers: Developers want to check the presence of code smells in the project.
Pre-Conditions	1. CodeNexus extension must be installed on the developer's Visual Studio Code. 2. CodeNexus extension must be activated on the developer's Visual Studio Code. 3. A project must be open in the Visual Studio Code workspace
Post-Conditions	Successful detection of the code smells in code if any are present
Main Success Scenario	1. The use case starts when the developer opens the project in visual studio code. 2. The developer opens the codenexus panel. 3. The developer selects the code smells he wants to detect. 4. The developer starts the detection. 5. The system collects all the relevant information needed for detection. 6. The system sends the collected data to the relevant microservice. 7. The system receives results from the microservice. 8. The system performs the final inter-file detection tasks. 9. The system displays the results of the detection tasks.
Extensions	4. A python file in the project is not compilable. 4.1. The system informs the user which file is not yet compilable. 4.2. The system skips detection tasks for the uncompilable code. 4. The specified code smells are not present in the code. 4.1. The system informs the user that the specified code smells are not present in the code base.

Table 2.3: UC-03

2.1.4 UC-04: Check Refactor Readiness

Use Case Name	Check refactor readiness
Scope	CodeNexus
Level	User Goals
Primary actor	Developers
Stakeholders and Interests	Developer: Developer wants to automatically refactor a code smell snippet.
Pre-Conditions	1. CodeNexus extension must be installed on the developer's Visual Studio Code. CodeNexus extension must be activated on the developer's Visual Studio Code. A project must be open in the Visual Studio Code workspace. Code smell must be detected in the workspace.
Post-Conditions	Successful refactoring of the select code snippet.
Main Success Scenario	1. The use case starts when code smell has been detected in a code snippet. 2. The system performs analysis on the code snippet and assigns it a readiness score. 3. Based on the readiness score, then the system determines if refactored code can be generated or not. 4. If the readiness score is high, the system makes the refactor option available to the developer.
Extensions	3. Refactor readiness score is low. 3.1. Refactor option not made available to the developer for that specific code snippet.

Table 2.4: UC-04

2.1.5 UC-05: Perform automated refactoring

Use Case Name	Perform automated refactoring
Scope	CodeNexus
Level	User Goals
Primary actor	Developers
Stakeholders and Interests	Developer: Developer wants to utilize the automated refactoring option to refactor problematic code snippets.
Pre-Conditions	1. CodeNexus extension must be installed on the developer's Visual Studio Code 2. CodeNexus extension must be activated on the developer's Visual Studio Code. 3. A project must be open in the visual studio code workspace. 4. Code smells have been detected in the project.
Post-Conditions	Successful refactoring of the select code snippet.
Main Success Scenario	1. The use case starts when the developer selects a code snippet where a code smell has been identified. 2. The developer checks if the refactor option is available. 3. If the refactor option is available, The developer selects the refactor option. 4. The system asks for confirmation of the refactor. 5. The user confirms the refactor 6. The system replaces the original code snippet with the refactored one.
Extensions	5. The user cancels the refactor. The system does not change the code snippet in any way. 2. There is no refactor option for the code snippet.

Table 2.5: UC-05

2.1.6 UC-06: Filter Code Smells

Use Case Name	Filter code smells
Scope	CodeNexus
Level	User Goals
Primary actor	Developers
Stakeholders and Interests	Developer: The developer wants the ability to filter out irrelevant code smells and focus only on the ones that are most important to him.
Pre-Conditions	1. CodeNexus extension must be installed on the developer's Visual Studio Code 2. CodeNexus extension must be activated on the developer's Visual Studio Code. 3. A project must be open in the visual studio code workspace. 4. Code smells have been detected in the project.
Post-Conditions	Successful filtering of code smell detection data.
Main Success Scenario	1. The use case starts when the developer wants to filter out a specific code smell type. 2. The system displays a list of the identified code smells. 3. The user then selects the code smells he wants to view. 4. The system filters out the detection data based on the selected code smells. 5. The filtered results are displayed to the developer in a prioritized order.
Extensions	2. No code smells have been detected in the code base. 2a. The system will inform the users that no code smells were found.

Table 2.6: UC-06

2.1.7 UC-07: Revert Code Changes

Use Case Name	Revert code changes
Scope	CodeNexus
Level	User Goals
Primary actor	Developers
Stakeholders and Interests	Developer: The developer wants to change a code snippet created through automated refactoring back to the original snippet.
Pre-Conditions	1. The developer must have a codenexus account. 2. The developer must be logged in to their account.
Post-Conditions	Selected snippet changed to its original state successfully.
Main Success Scenario	1. The use case starts when the developer wants to change an automatically refactored code snippet back to its original state. 2. The user selects the code snippet. 3. The user selects the revert option. 4. The system retrieves the relevant data. 5. The system performs readiness assessment on the selected code snippet. 6. The system replaces the selected snippet with the original snippet.
Extensions	6. The code snippet fails the revert readiness assessment. 6.1. The system display notifies the user. 6.2. The code snippet remains unchanged,

Table 2.7: UC-07

2.1.8 UC-08: Configure Custom Rulesets

Use Case Name	Configure custom rulesets
Scope	CodeNexus
Level	User Goals
Primary actor	Developers
Stakeholders and Interests	Developer: The developer wants to customize the detection and refactoring of code smells for a certain project.
Pre-Conditions	1. CodeNexus extension must be installed on the developer's visual studio code. 2. CodeNexus extension must be activated on the developer's visual studio code. 3. A project must be open in the Visual Studio Code workspace.
Post-Conditions	Selected snippet changed to its original state successfully.
Main Success Scenario	1. The use case starts when the developer wants to set up rulesets for a specific project. 2. The developer opens the JSON rulesets file. 3. The developer edits the file and saves the configuration details. 4. The system scans the file and applies the defined configuration.
Extensions	4. The file has invalid configurations. 4.1. The system notifies the user of the error.

Table 2.8: UC-08

2.1.9 UC-09: View Codebase Logs

Use Case Name	View codebase logs
Scope	CodeNexus
Level	User Goals
Primary actor	User
Stakeholders and Interests	Users: Users want to view and analyze the code smell detection and refactoring history for one of the projects.
Pre-Conditions	1. The user must have a codenexus account. 2. The user must be logged in to their account. 3. The user must have access to the project.
Post-Conditions	Display the entire history code smells within the user's project.
Main Success Scenario	1. The use case starts when the user wants to view the code smells detection and refactoring history for a project . 2. The user selects the project from the list of his projects. 3. The system displays project details. 4. The user selects the view logs option. 5. The system displays the logs for the selected project.
Extensions	5. There are no logs for the selected project. 5.1. The system notifies the user that no logs are available.

Table 2.9: UC-09

2.1.10 UC-10: Share Codebase Logs

Use Case Name	Share codebase logs
Scope	CodeNexus
Level	User Goals
Primary actor	User
Stakeholders and Interests	Users: Users want to share logs of a project with others.
Pre-Conditions	1. The user must have a codenexus account. 2. The user must be logged in to their account. 3. The user must have access to the project.
Post-Conditions	Message response from the microservice.
Main Success Scenario	1. The use case starts when the developer wants to share his project logs with someone. 2. The developer goes to the share option. 3. The system shows a form to enter user credentials. 4. The developer enters the username or email of the other person in the given space. 5. The system checks if a user with the provided details exists. 6. If the user exists, the system adds the user to the project.
Extensions	4. The developer enters an invalid username/email. 4.1. The system provides the developer with two options: to either re-enter user credentials or cancel the action. 6. If the user being added does not exist, the system notifies the developer. 6.1. The system provides the developer with two options: to either add the user or cancel the action.

Table 2.10: UC-10

2.1.11 UC-11: Isolated Compilation and Testing of Refactored Code

Use Case Name	Isolated compilation and testing of refactored code
Scope	CodeNexus
Level	User Goals
Primary actor	Developer
Stakeholders and Interests	Developer: Developer wants to test the code refactorings in an isolated environment before allowing the extension to update the code snippets within the actual project.
Pre-Conditions	1. The developer must have a codenexus account. 2. The developer must be logged in to their account. 3. The developer must have initiated some code refactoring.
Post-Conditions	The refactored code is mapped onto the project.
Main Success Scenario	1. The use case starts when the system provides the user with refactored code. 2. The system will display the option to test the refactored code. 3. The developer selects the test option. 4. The system creates a clone project with the refactored changes. 5. The system provides user with a terminal to test the functionality of the code. 6. The user can manually test the code functionality. 7. The user accepts the refactored changes. 8. The system maps the refactored changes from the clone project onto the actual project. 9. The system discards the clone project.
Extensions	4 The refactored code is not compilable. 4.1 The system will discard the code refactoring. 7 User rejects the code refactoring changes 7.1 The system will discard the code refactorings 7.2 The system discards the clone project.

Table 2.11: UC-11

2.1.12 UC-12: Visualizing Inter-File Dependencies

Use Case Name	Visualizing inter-file dependencies
Scope	CodeNexus
Level	User Goals
Primary actor	User
Stakeholders and Interests	User: Users want to analyze how different files in the project are related to one another.
Pre-Conditions	1. The user must have a codenexus account. 2. The user must be logged in to their account.
Post-Conditions	Display the dependency visualization of the project.
Main Success Scenario	1. The use case starts when the user wants to view how changes in one part of the code base can affect other components. 2. The user opens the project. 3. The system displays project details. 4. The user goes to the impact analysis page. 5. The system displays a dependency visualization of the project.
Extensions	5. Only one or no python files present in the project. 5.1. There will be no visual representation of dependencies

Table 2.12: UC-12

2.2 Functional Requirements

2.2.1 Module 1 - Codebase Analysis

1. Project Structure Extraction

- 1.1.1 The system shall scan the active project folder and subfolders to identify all .py files.
- 1.1.2 The system shall only process files with the .py extension for analysis and content extraction.
- 1.1.3 The system shall structure the extracted content in a format that allows for further analysis.

2. Abstract Syntax Tree Generation

- 1.2.1 The system shall generate an AST for each Python file.
- 1.2.2 The system shall automatically re-generate the AST for a file whenever project is saved.
- 1.2.3 The system shall indicate whenever AST generation for a specific file fails.
- 1.2.4 The system shall restructure the AST to ensure further processing.

3. Code Dependency Graph Generation

- 1.3.1 The system shall create an inter-file dependency graph by analyzing the codebase to identify how different files are related to each other.
- 1.3.3 The system shall update the inter-file dependency graph whenever the AST is updated.
- 1.3.4 The system shall represent files as nodes of the dependency graph.
- 1.3.5 The system shall represent inter-file dependencies as directed edges in the dependency graph.
- 1.3.6 The system shall ensure that the graph accurately reflects the current state of the codebase, including all inter-file connections and dependencies.

2.2.2 Module 2 - Detection Module

1. Real-Time Detection of Code Smells

- 2.1.1 The system shall scan all the Python files in the workspace for code smells after regular intervals.
- 2.1.2 The system shall run detection on all the scanned files for any present code smells.
 - i. The system shall detect occurrences of magic numbers within the codebase.
 - ii. The system shall detect unutilized variables within the codebase.
 - iii. The system shall detect occurrences of dead code within the codebase.
 - iv. The system shall detect functions or methods with long parameter lists.
 - v. The system shall detect conflicts caused by the use of global variables.
 - vi. The system shall detect overly complex conditional statements within the codebase.
 - vii. The system shall detect occurrences of too many nested loops within the codebase
 - viii. The system shall detect occurrences of unreachable code within the codebase.

- ix. The system shall detect excessive use of flags within the codebase.
- x. The system shall detect violations of variable naming conventions within the codebase.
- xi. The system should detect inconsistent naming within the codebase.
- xii. The system should detect non-descriptive method names within the codebase

2.1.3 The system shall allow the user to view identified code smells.

2. User Triggered Detection of Code Smells

2.2.1 The system shall allow the user to manually trigger the detection of certain code smells.

- i. The system shall allow the user to manually trigger the detection of long functions.
- ii. The system shall allow the user to manually trigger the detection of switch statement abuser.
- iii. The system shall allow the user to manually trigger the detection for tight coupling.
- iv. The system shall allow the user to manually trigger the detection for god object.
- v. The system shall allow the user to manually trigger the detection of inconsistent naming.
- vi. The system shall allow the user to manually trigger the detection of feature envy.
- vii. The system should allow the user to manually trigger the detection of code fragments that have been commented out.
- viii. The system shall allow the user to manually trigger the detection temporary field.
- ix. The system should allow the user to manually trigger the detection over-encapsulation.
- x. The system shall allow the user to manually trigger the detection middle man.

2.2.2 The system shall allow the user to view the problematic code snippets within the file.

2.2.3 The system shall enable the user to view details about the issue detected in the selected problematic code snippet.

2.2.3 Module 3 - Refactoring Module

3.1 Refactoring Readiness Assessment of Identified Code Smells

- 3.1.1 The system shall evaluate the refactoring readiness of the code.
 - i. The system shall evaluate the complexity of each identified code smell.
 - ii. The system shall evaluate the dependencies, as well as the impact of changes associated with each identified code smell.
- 3.1.2 The system shall use the dependency graph to determine files that could be affected by refactoring.

3.3 Refactoring of Identified Code Smells

- 3.2.1 The system shall allow the user to manually refactor the code.
- 3.2.2 If the code passes the refactoring readiness assessment, then the system shall allow the user to use the automatic refactoring option to fix their code.
- 3.2.3 The system shall allow the user to analyze the changes made through refactoring.

3.3 Filter and Refactor Specific Code Based on Detected Code Smells

- 3.3.1 The system shall allow the user to filter out which code smells to refactor.

3.4 Cascading Code Refactoring

- 3.4.1 The system shall propagate any changes made during the automated refactoring process.
- 3.4.2 The system shall ensure consistency in variable and function names as well as any other relevant code structures across the entire codebase.

3.5 Isolated Compilation and Testing of Refactored Code Base

- 3.5.1 The system shall allow the user to do a test run of the refactored codebase.
- 3.5.2 The system shall run the refactored code base in an isolated environment.
- 3.5.3 The system shall allow the user to validate the functionality of the refactored code base.

3.6 Isolated Compilation and Testing of Refactored Code Snippet (Supplementary)

- 3.6.1 The system shall allow the user to do a test run of the refactored code.
- 3.6.2 The system shall run the refactored code snippet in an isolated environment.

3.6.3 The system shall display the output of both the refactored and the original code snippets.

3.6.4 The system shall compare the outputs of both the refactored and the original code snippets.

3.7 Personalized Refactoring Suggestions (Supplementary)

3.7.1 The system shall analyze user coding practices.

3.7.2 The system shall provide refactoring suggestions according to users coding practices and style.

2.2.4 Module 4 - Change Logs Module

4.1 Code Smell Detection and Refactoring Tracker

4.1.1 The system shall track all the code smells identified in the detection process.

4.1.2 The system shall track all the automated refactors.

4.1.3 The system shall maintain a detailed change logs of detection and refactoring.

4.1.4 The system shall track the current status of each code smell and update it as changes occur.

4.1.5 The system should allow users to manually update the status of a code smell.

4.1.6 The system shall continuously track the changes in inter-file dependencies caused by refactoring.

4.1.7 The system shall allow users to generate reports of change logs.

4.2 File Dependency Tracker

4.2.1 The system shall track inter-file dependencies within the project.

4.2.2 The system shall maintain a history of the inter-file dependencies.

4.3 Revert Readiness Assessment

4.3.1 The system shall perform a revert readiness assessment before trying to revert a refactored code snippet.

4.3.2 The system shall analyze the impact of the revert across the project.

4.3.3 If the user has manually modified the automatically refactored code snippets, then the system shall prevent reverting code changes.

4.3.5 If no issues are detected., then the system shall allow the user to proceed with the revert.

4.3.6 If a revert is not possible due to problems, then the system shall notify the user.

4.4 Code Rollback

4.4.2 The system shall replace the refactored code with the original code when revert is triggered.

4.4.3 The system shall ensure that all changes are reverted consistently across the codebase.

4.4.4 The system shall notify the user upon successful completion of revert.

2.2.5 Module 5 - Rulesets Module

5.1 VS Code Extension Rulesets

5.1.1 The system shall allow the user to filter out which code smells to detect.

5.1.2 The system shall allow the user to filter out which files to consider during detection.

5.1.3 The system shall allow the user to filter out which code smells to refactor automatically.

5.1.4 The system shall allow the user to filter out which files to refactor automatically.

2.2.6 Module 6 - Website Development Module

6.1 The system shall allow the user to create an account for codeNexus.

6.2 The system shall allow the user to login to their account using username and password.

6.3 The system shall allow the user to link their account with the vscode extension.

6.4 Shareable Logs Access

6.4.1 The system shall allow project owners to invite viewers to their project.

6.4.2 The system shall allow project owners to view all the viewers of their project.

6.4.3 The system shall allow viewers to view project owners and other viewers of a project.

6.4.4 The logs shall display detailed information about each identified code smell.

6.4.5 The system shall allow users to filter logs by specific code smells, file locations, and dates.

- 6.4.6 The system shall maintain records of all refactoring actions taken on code smells.

6.5 Project Visualization

- 6.5.1 The system shall allow the user to view all the files within a project.
- 6.5.2 The system shall allow the user to visualize the folder structure of a project.
- 6.5.3 The system shall allow the user to visualize the inter-file dependencies present in a project.
- 6.5.4 The system shall visualize how multiple files in the codebase are related.

6.6 Inter-File Impact Analysis

- 6.6.1 The system shall allow the user to view file dependencies by showing how modifications in methods, classes, or variables in one file may impact other files in the codebase.

6.7 Code Smells Timeline

- 6.7.1 The system shall display a history of code smells detected over time.
- 6.7.2 The system shall display the number of detections made during each scan.
- 6.7.3 If a code smell has been refactored automatically, then the system shall display the modified code snippet.
- 6.7.4 The system should utilize graphs to show how the number of code smells has evolved in a code base over time.

2.3 Non-Functional Requirements

2.3.1 Reliability

- REL-1 The system shall log all failures and errors.
- REL-2 The system shall handle unexpected errors without losing user progress or critical information.
- REL-3 Code reversion features shall restore the code to its original state without causing any additional issues or inconsistencies.
- REL-4 Real-time detection shall update as code changes occur, ensuring that the most recent state of the codebase is always analyzed.
- REL-5 Logs and change history shall be recorded and retrievable at any time for review.

2.3.2 Usability

- USE-1 The visual markers and "Problems" tab interface should be easy to interpret and navigate.
- USE-2 The system shall allow customization of code smell detection and refactoring rules for flexibility based on different project requirements.
- USE-3 Real-time detection shall not disrupt the coding experience, and the results should be displayed as the developer edits code.

2.3.3 Performance

- PER 2 Log generation and updates in the database shall occur without slowing down the active code analysis and refactoring processes.
- PER 3 The LLM Service should provide suggestions and assessments in a timely manner.

2.3.4 Security

- SEC-1 The system shall restrict unauthorized access to project logs.
- SEC-2 The system shall automatically log out users after a one week of inactivity.
- SEC-3 The extension shall allow access only to the open and active project folder and shall not scan files or directories outside the user's designated workspace.
- SEC-4 User actions, including code smell detection and refactoring, shall be logged securely to prevent tampering or data loss.

2.3.5 Scalability

- SCA-1 The detection service should be able to handle large files without performance issues.
- SCA-2 The system shall handle large codebases with multiple files and folders.
- SCA-3 The extension shall manage multiple user-triggered detection requests, maintaining smooth functionality for both single-file and project-wide analysis.
- SCA-4 The Changelogs Service shall scale to accommodate an extensive history of code changes, maintaining performance in log retrieval and filtering.

Chapter 3

System Overview

3.1 Architectural Design

The architecture includes two client interfaces: a VS Code extension and a web application, which interact with an API Gateway. The gateway is connected to a load balancer provided by Amazon AWS, responsible for distributing incoming traffic efficiently. The API Gateway routes requests to one of four microservices: Code Smell Detection, Refactoring, Refactoring Readiness, and Changelogs.

Among these microservices, the Changelogs service requires authentication for access, while the Code Smell Detection, Refactoring, and Refactoring Readiness services do not require authentication. However, the Detection and Refactoring microservices have the capability to interact with the Changelogs microservice to store and retrieve data. For these interactions, they must pass user authentication to gain access to changelog data.

The Changelogs microservice has access to a MongoDB database to store and manage changelog information. It acts as the central hub for all data related tasks across the system.

In addition to these microservices, there is an LLM (Large Language Model) microservice that performs advanced processing tasks. This microservice uses Amazon SQS to manage task queues, where Detection and Refactoring services send their tasks. Once a task is processed by the LLM, the completed response is added to a message queue, which the API Gateway accesses to send corresponding messages back to the appropriate client sockets.

To aid its operations, the LLM microservice uses a VectorDB, enabling vector retrieval for tasks requiring advanced data processing or contextual information. It also uses a cache to store frequently accessed vectors, improving response times and reducing redundant data fetching.

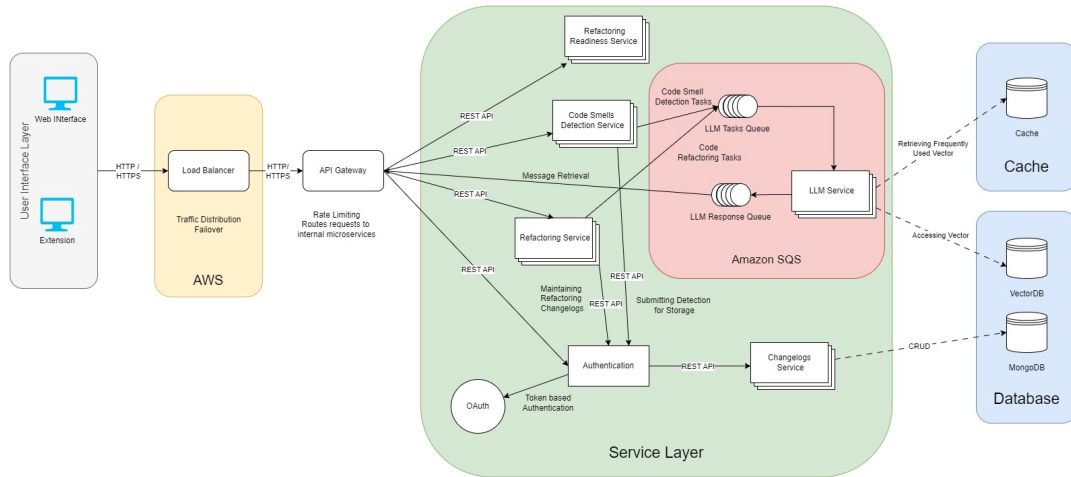


Figure 3.1: Architecture of CodeNexus

The architecture efficiently distributes traffic through the AWS load balancer and manages the routing and processing of various requests through a combination of authentication checks, microservices, and an task-processing system with the LLM microservice and Amazon SQS.

3.2 Design Models

- Activity Diagram

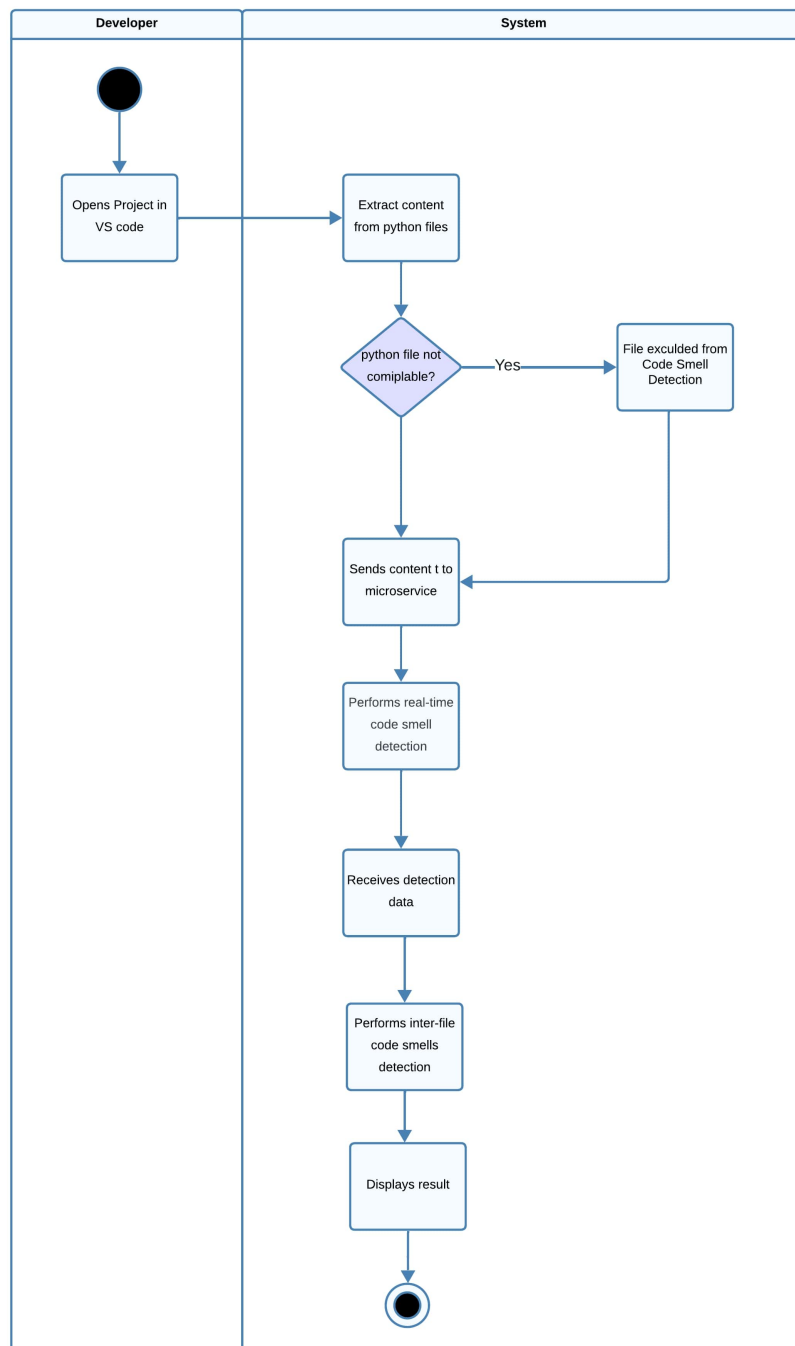


Figure 3.2: UC-01 Activity Diagram

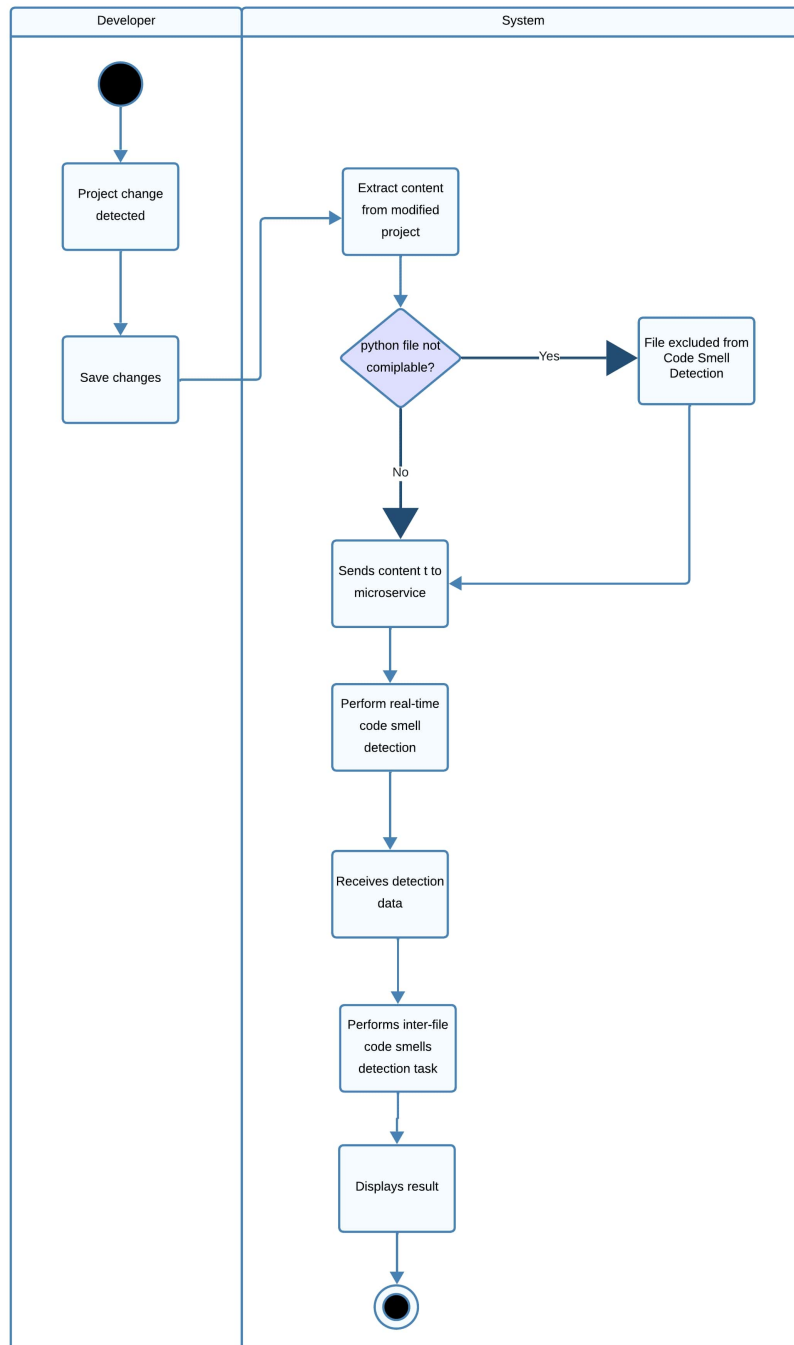


Figure 3.3: UC-02 Activity Diagram

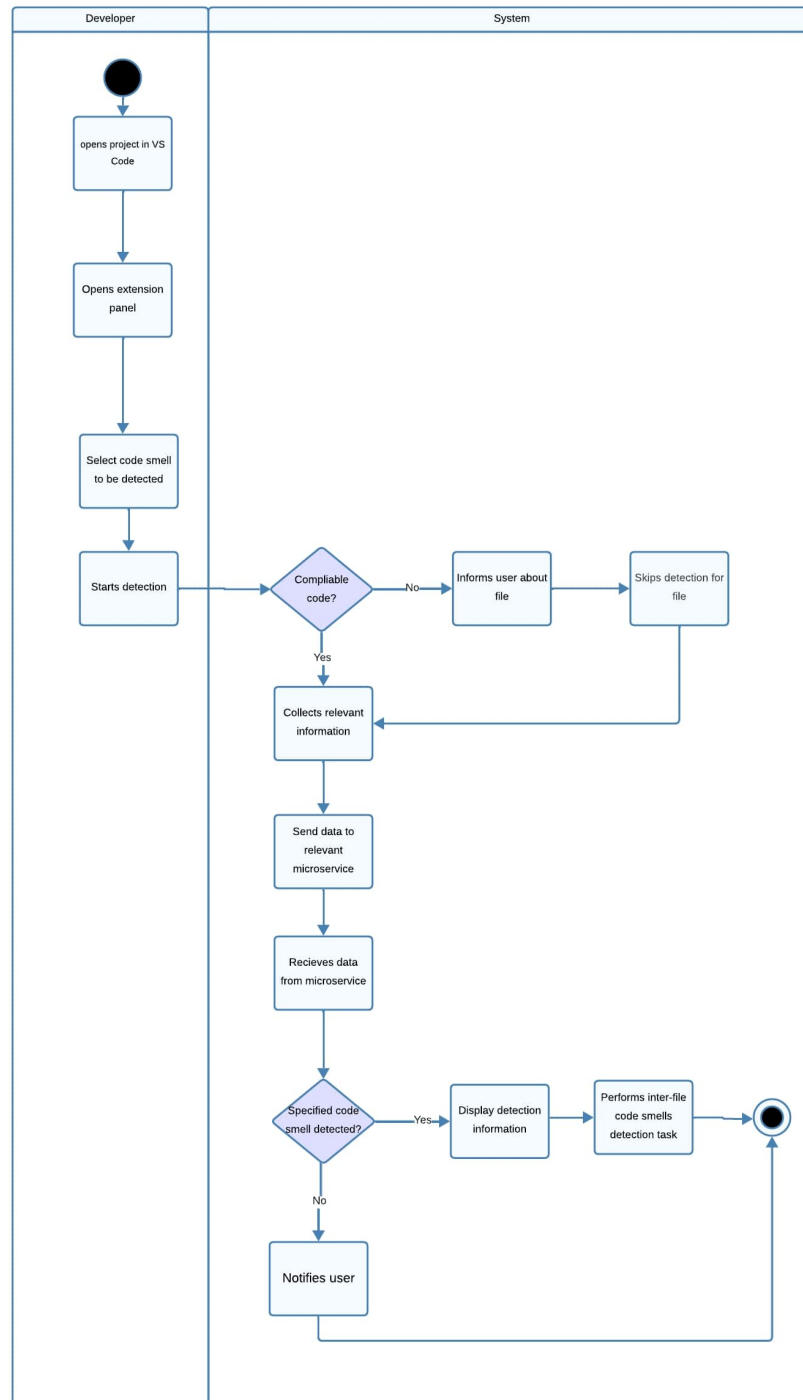


Figure 3.4: UC-03 Activity Diagram

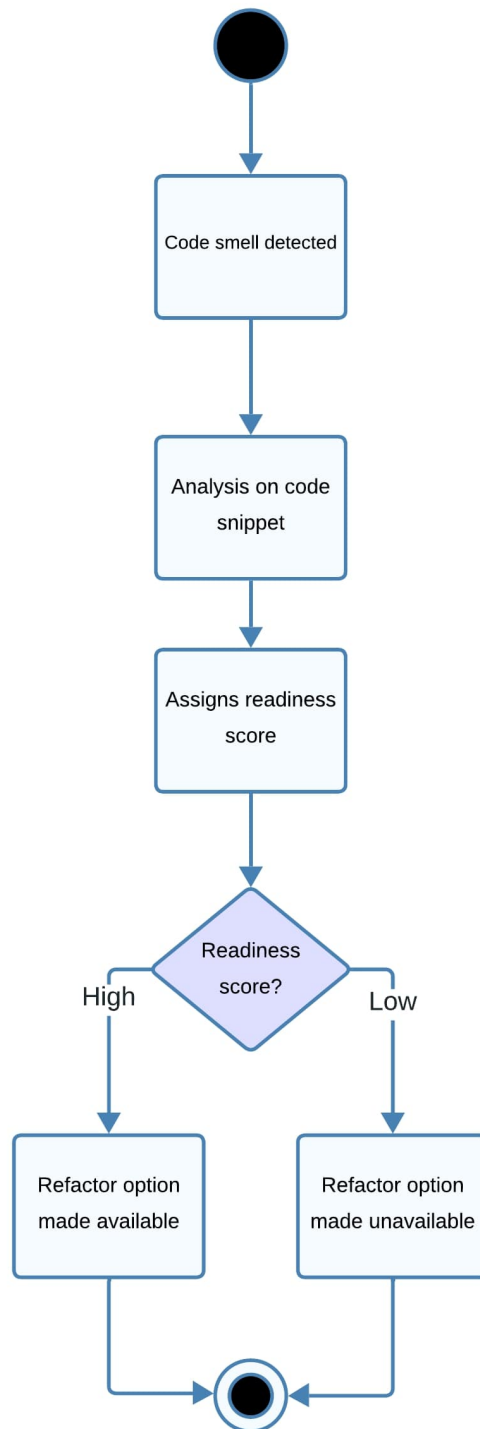


Figure 3.5: UC-04 Activity Diagram

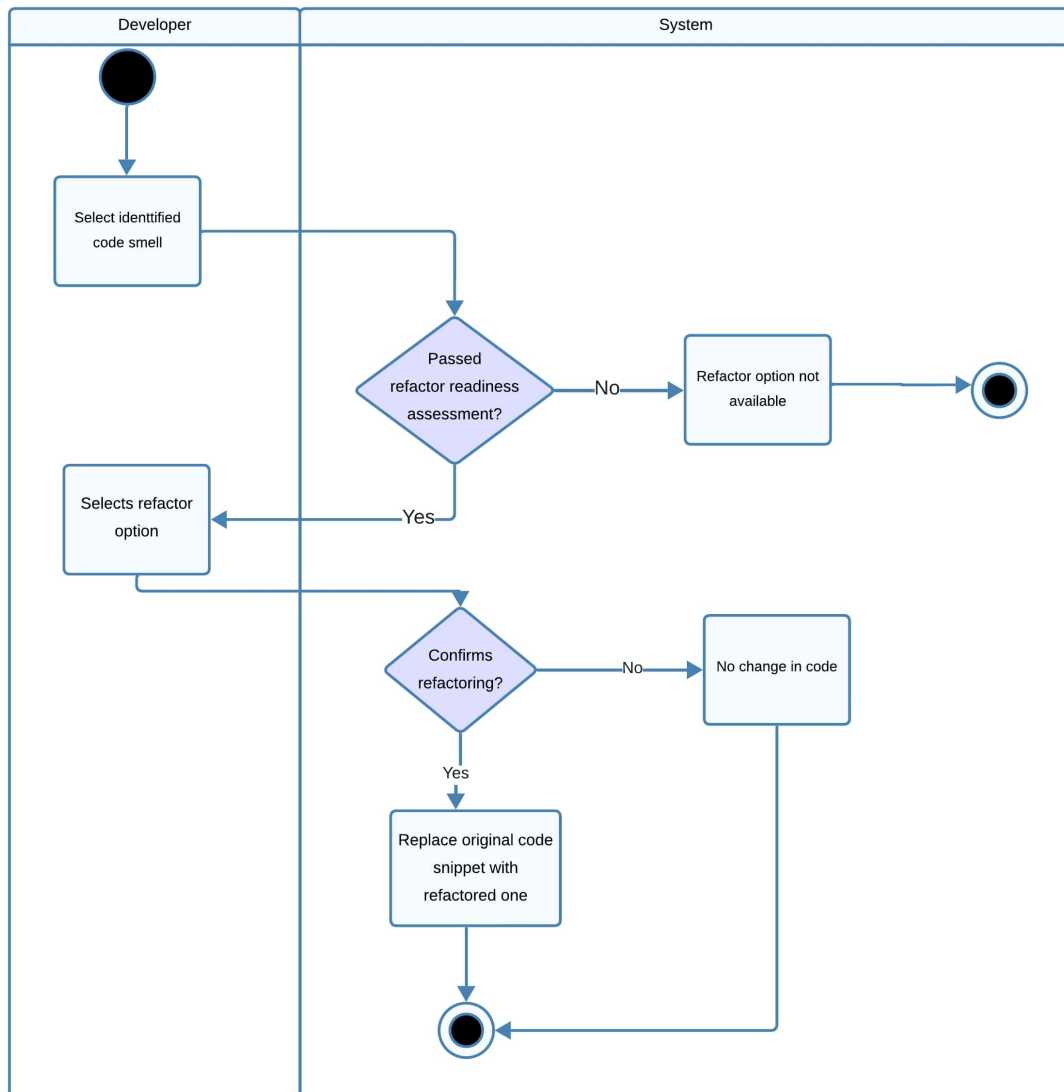


Figure 3.6: UC-05 Activity Diagram

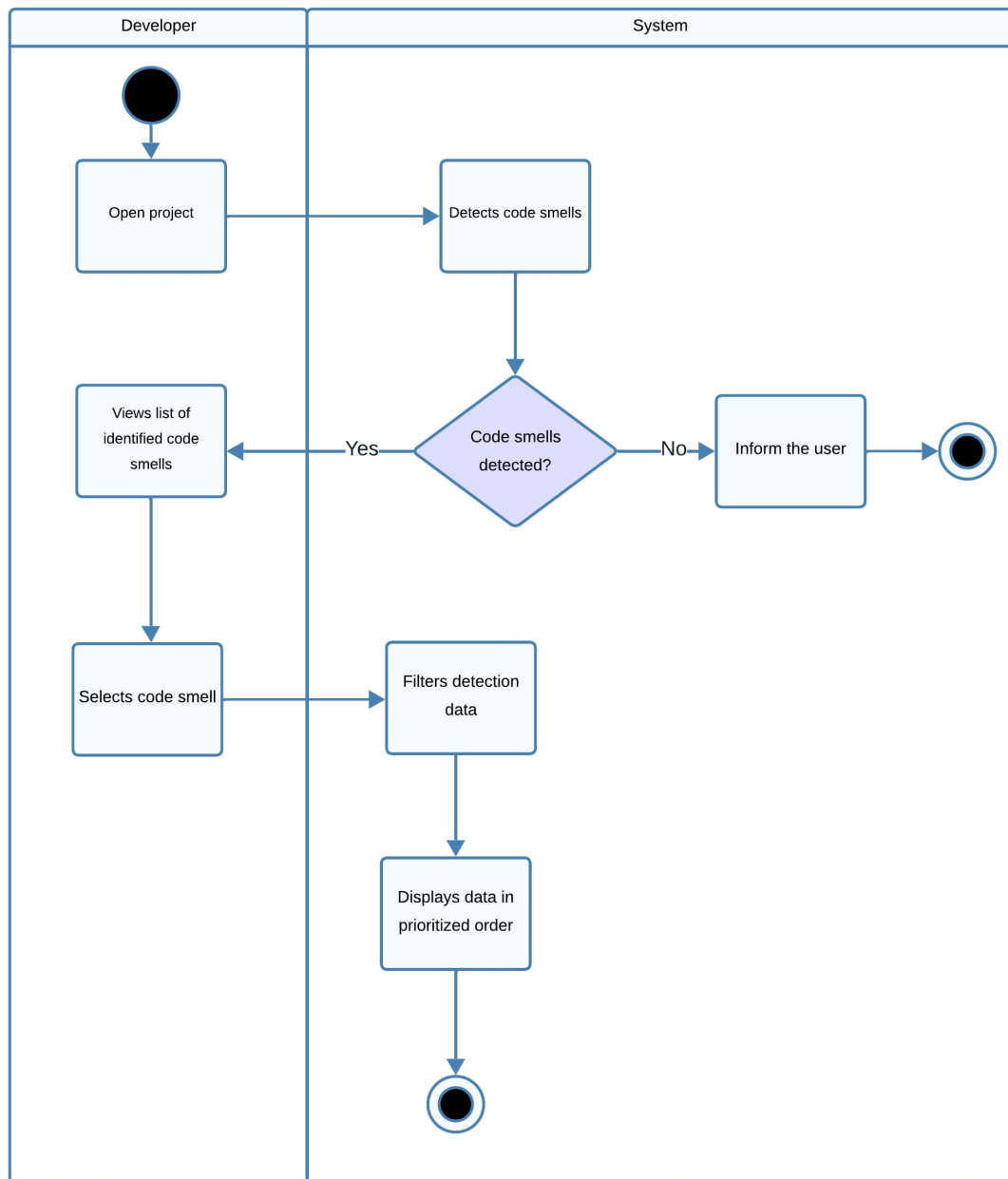


Figure 3.7: UC-06 Activity Diagram

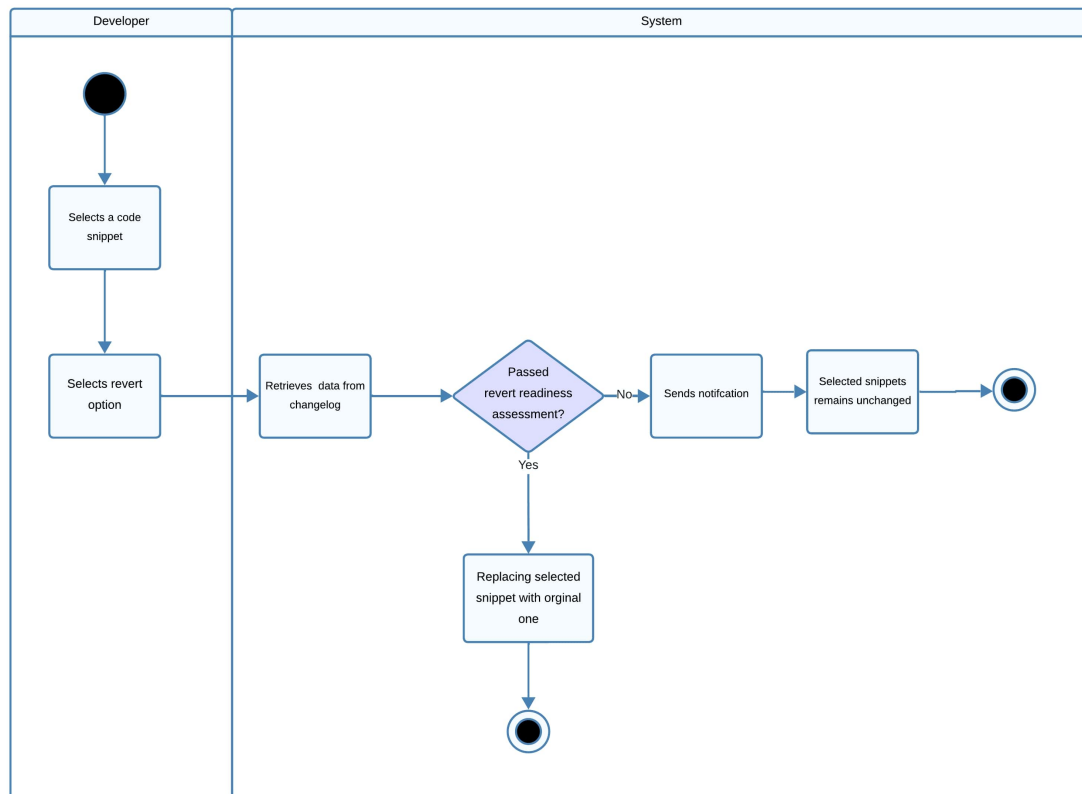


Figure 3.8: UC-07 Activity Diagram

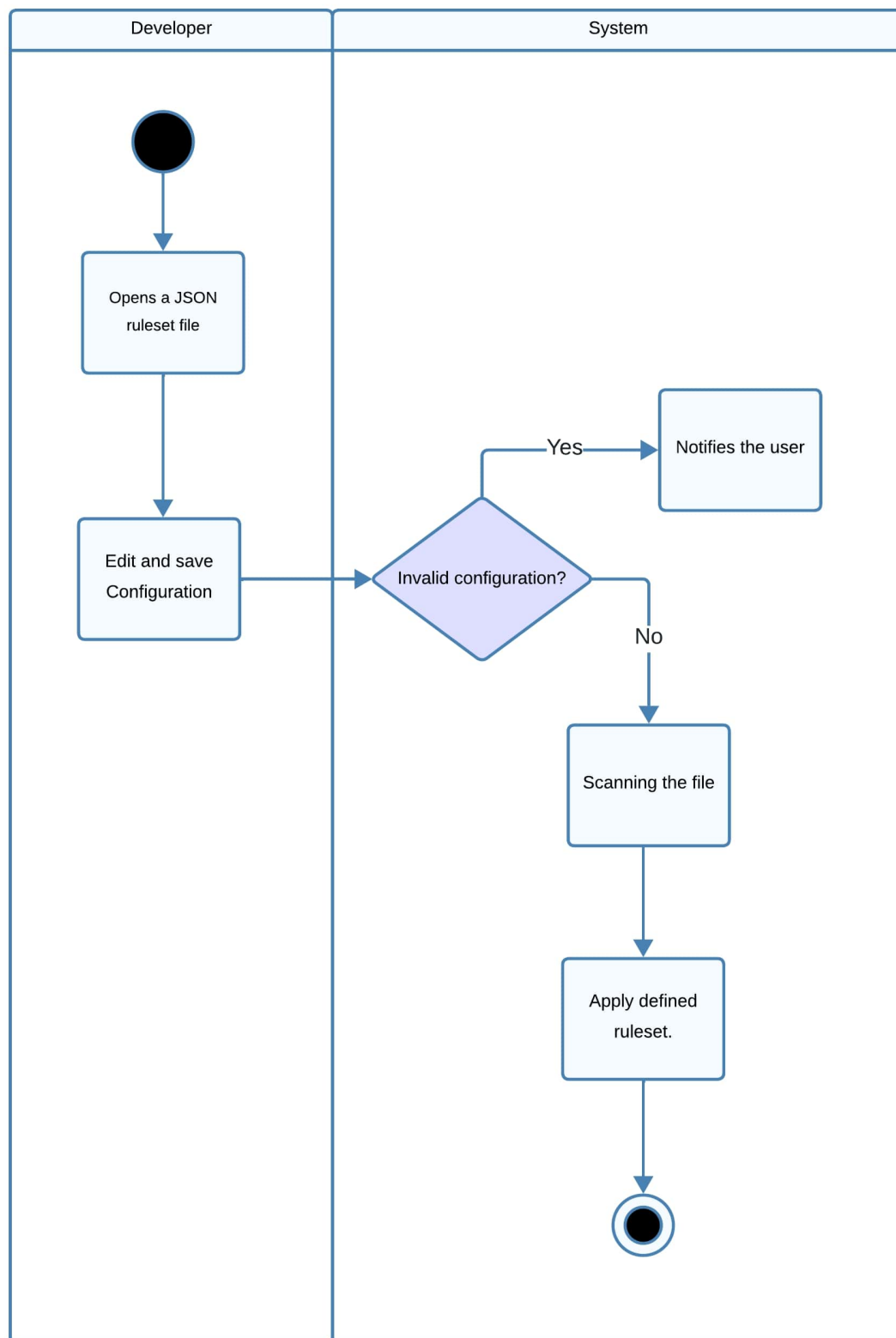


Figure 3.9: UC-08 Activity Diagram

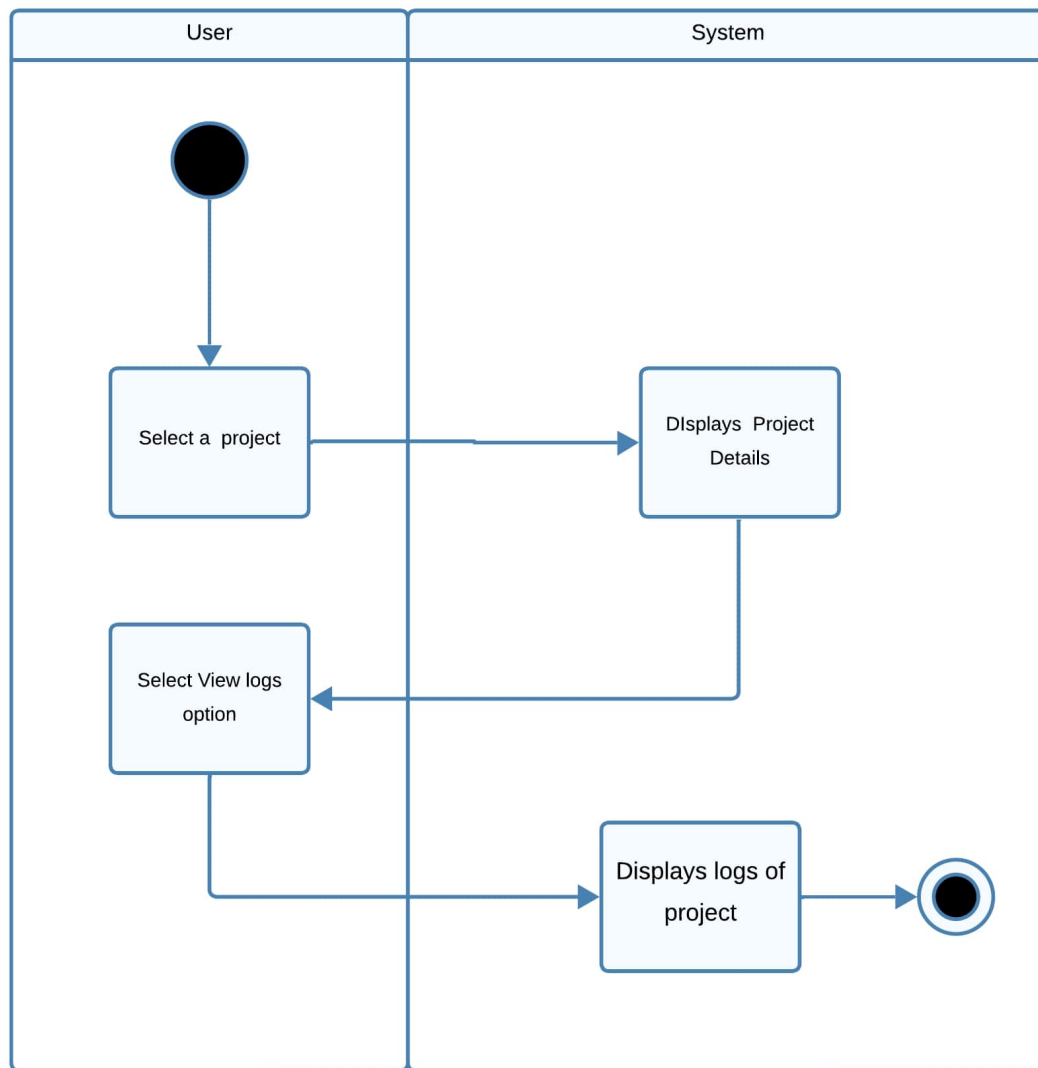


Figure 3.10: UC-09 Activity Diagram

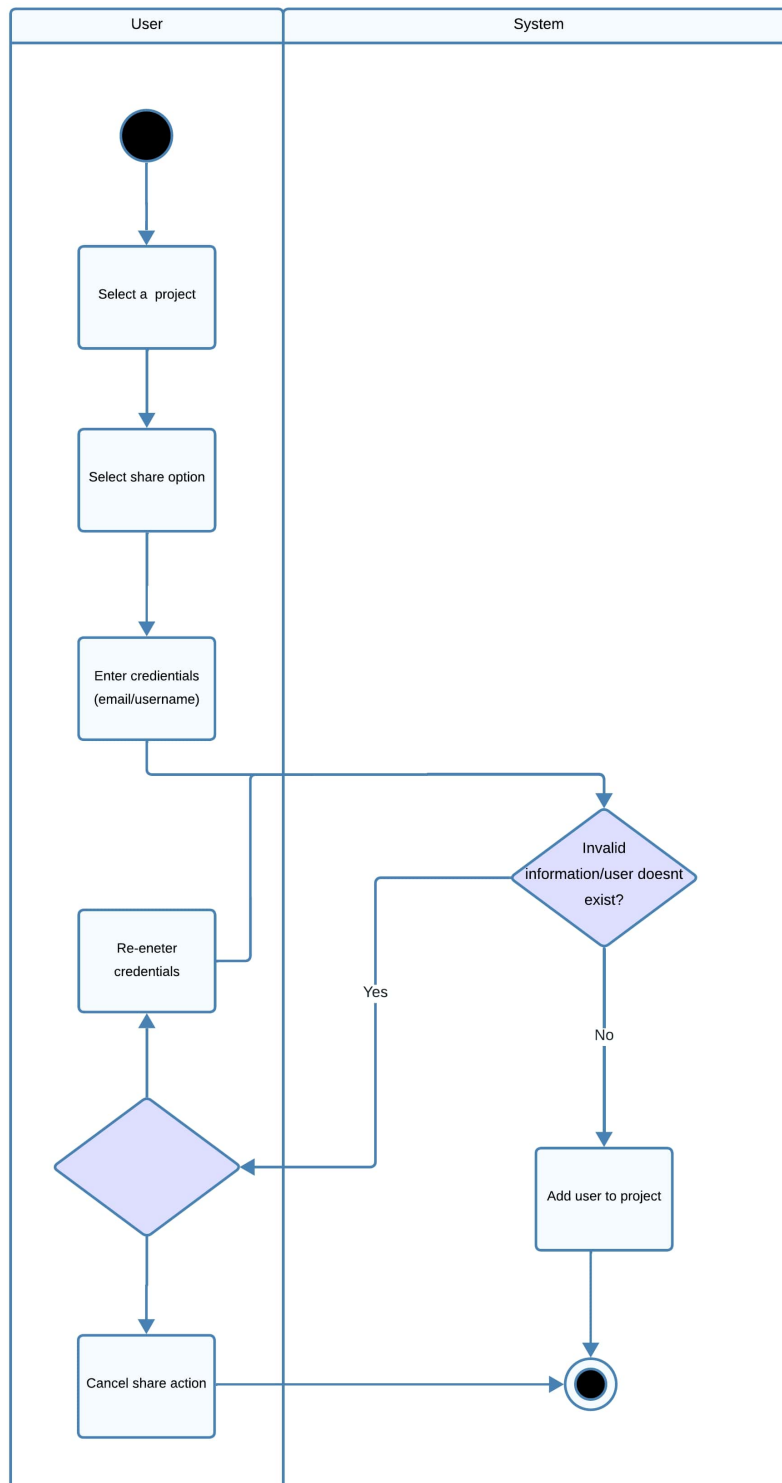


Figure 3.11: UC-10 Activity Diagram

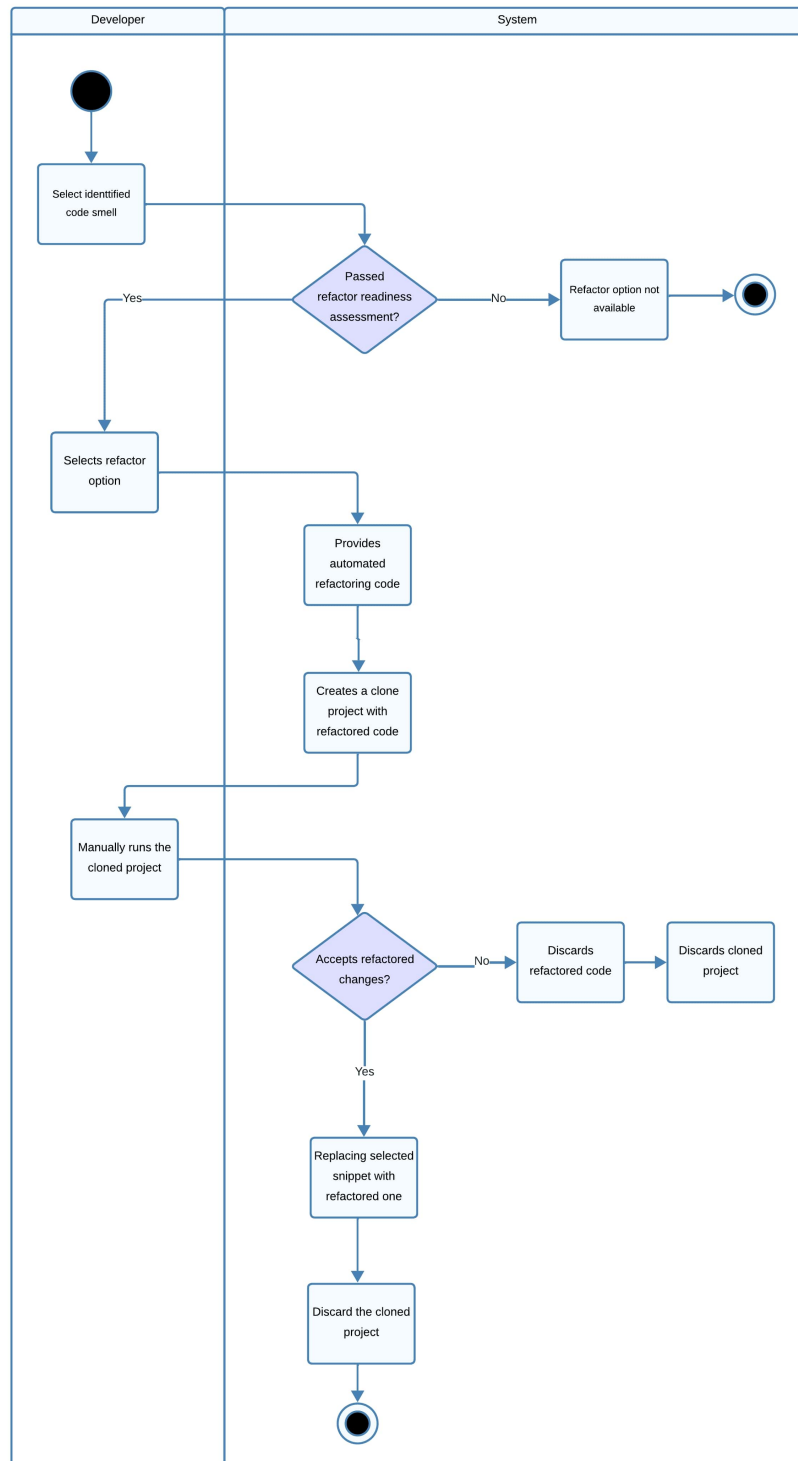


Figure 3.12: UC-11 Activity Diagram

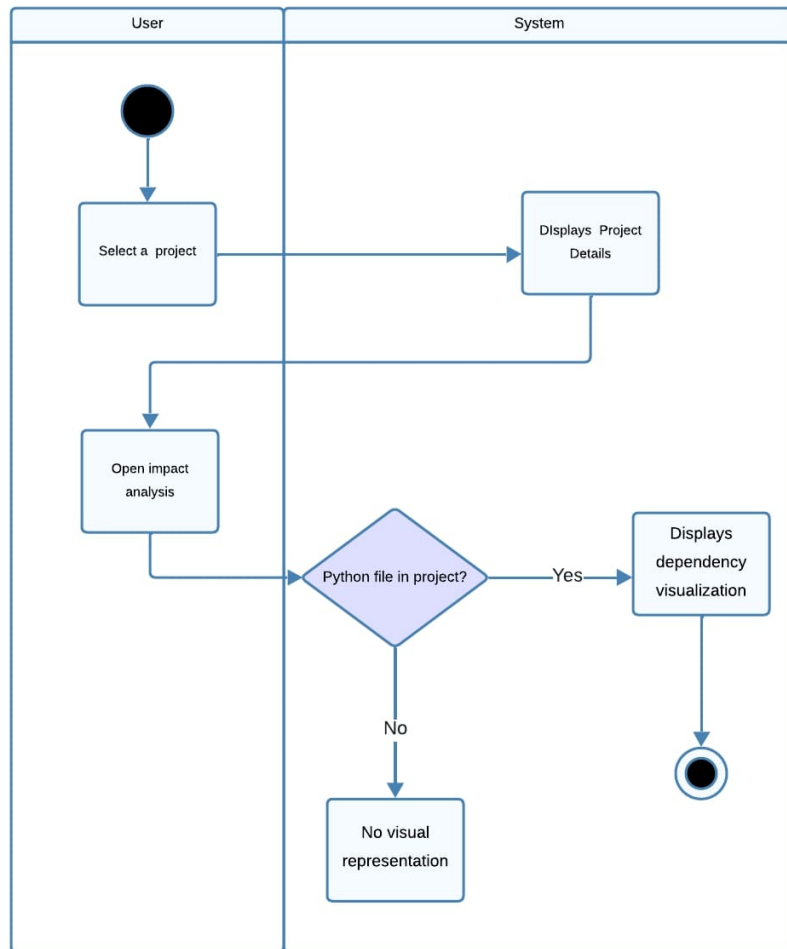


Figure 3.13: UC-12 Activity Diagram

- Data Flow Diagram

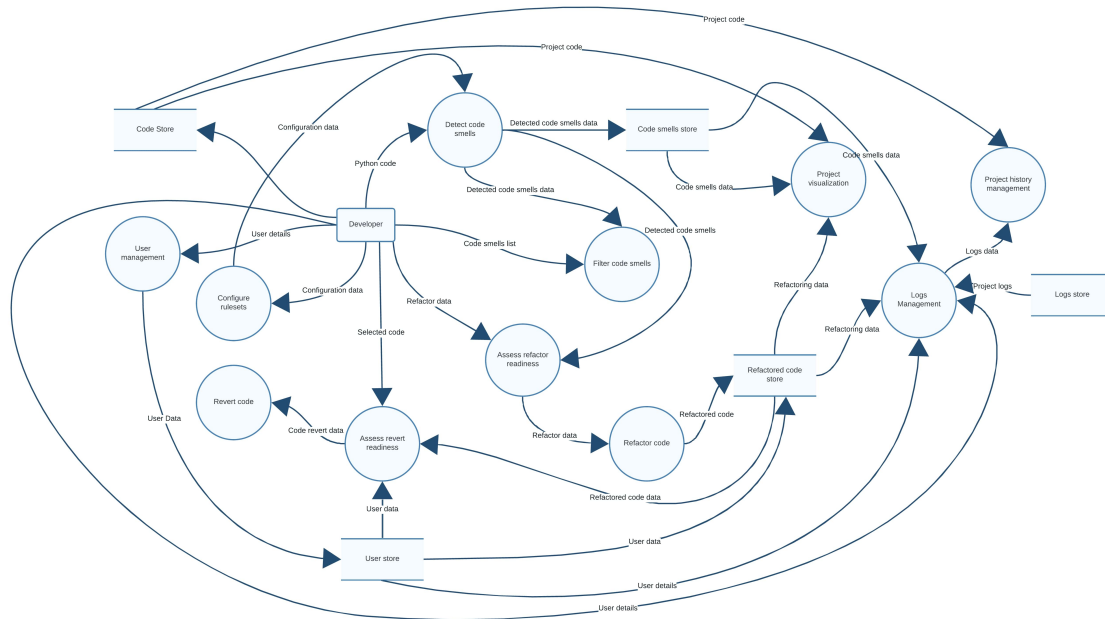


Figure 3.14: Data Flow Diagram

- System-level Sequence Diagram

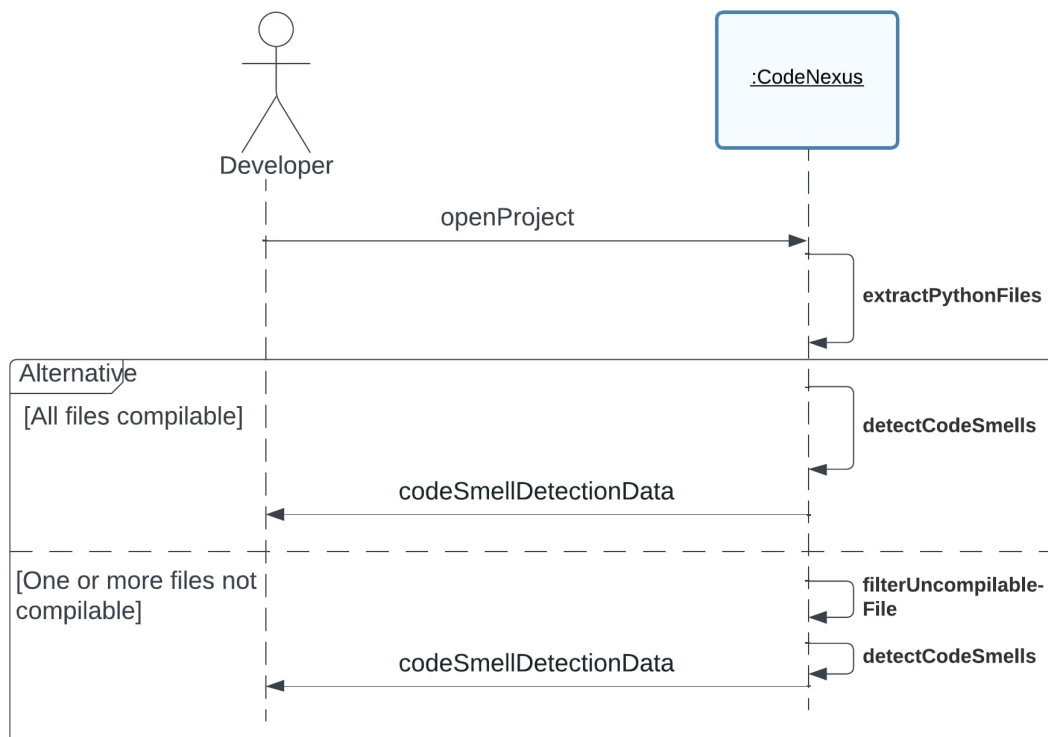


Figure 3.15: UC-01: System Sequence

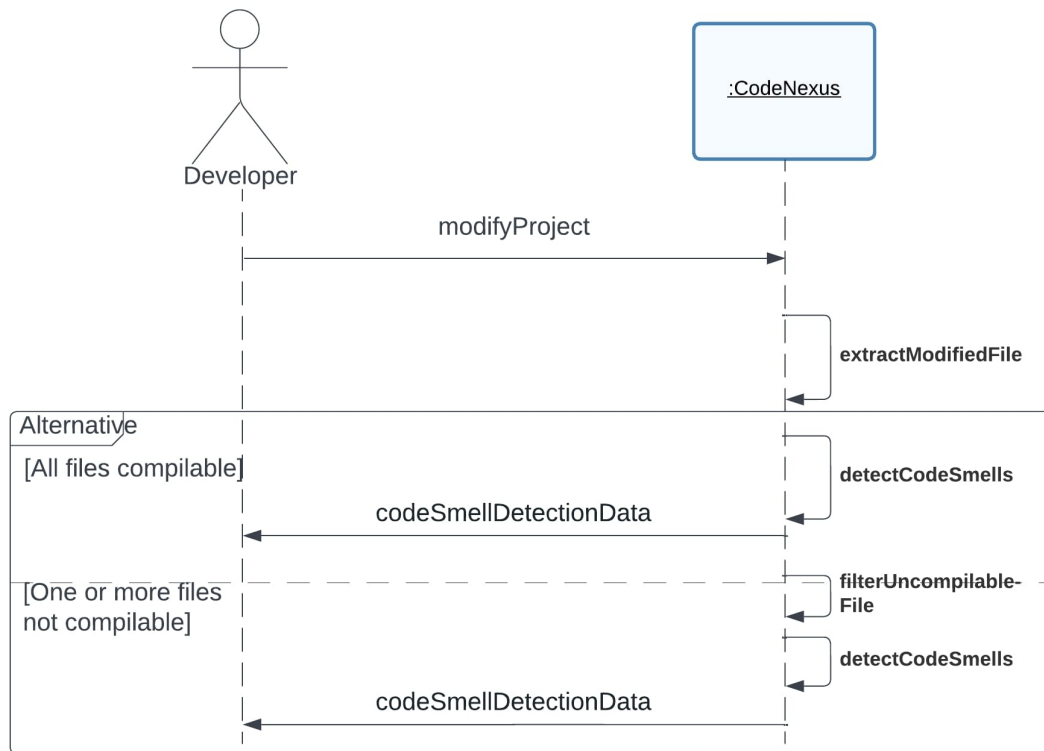


Figure 3.16: UC-02: System Sequence

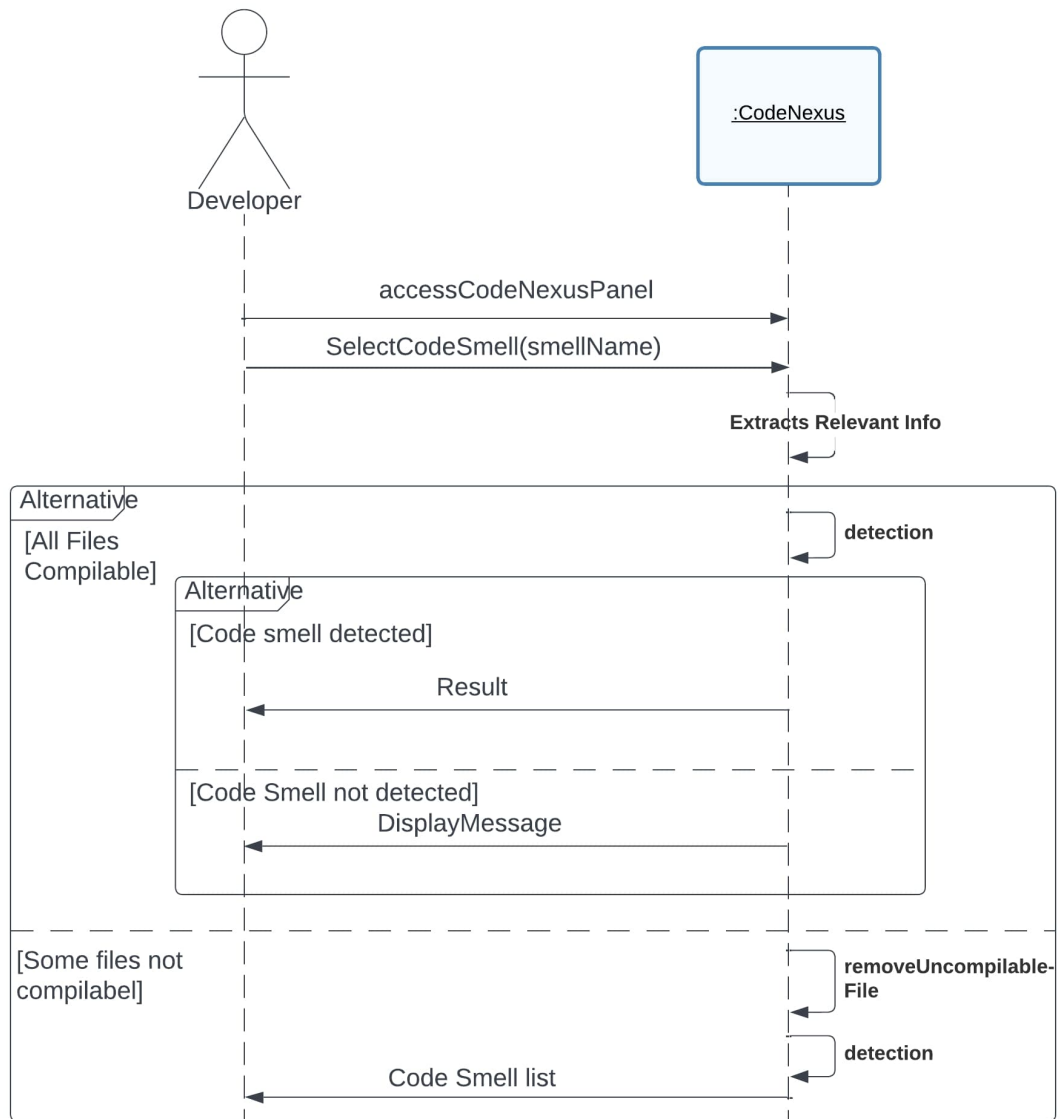


Figure 3.17: UC-03 System Sequence

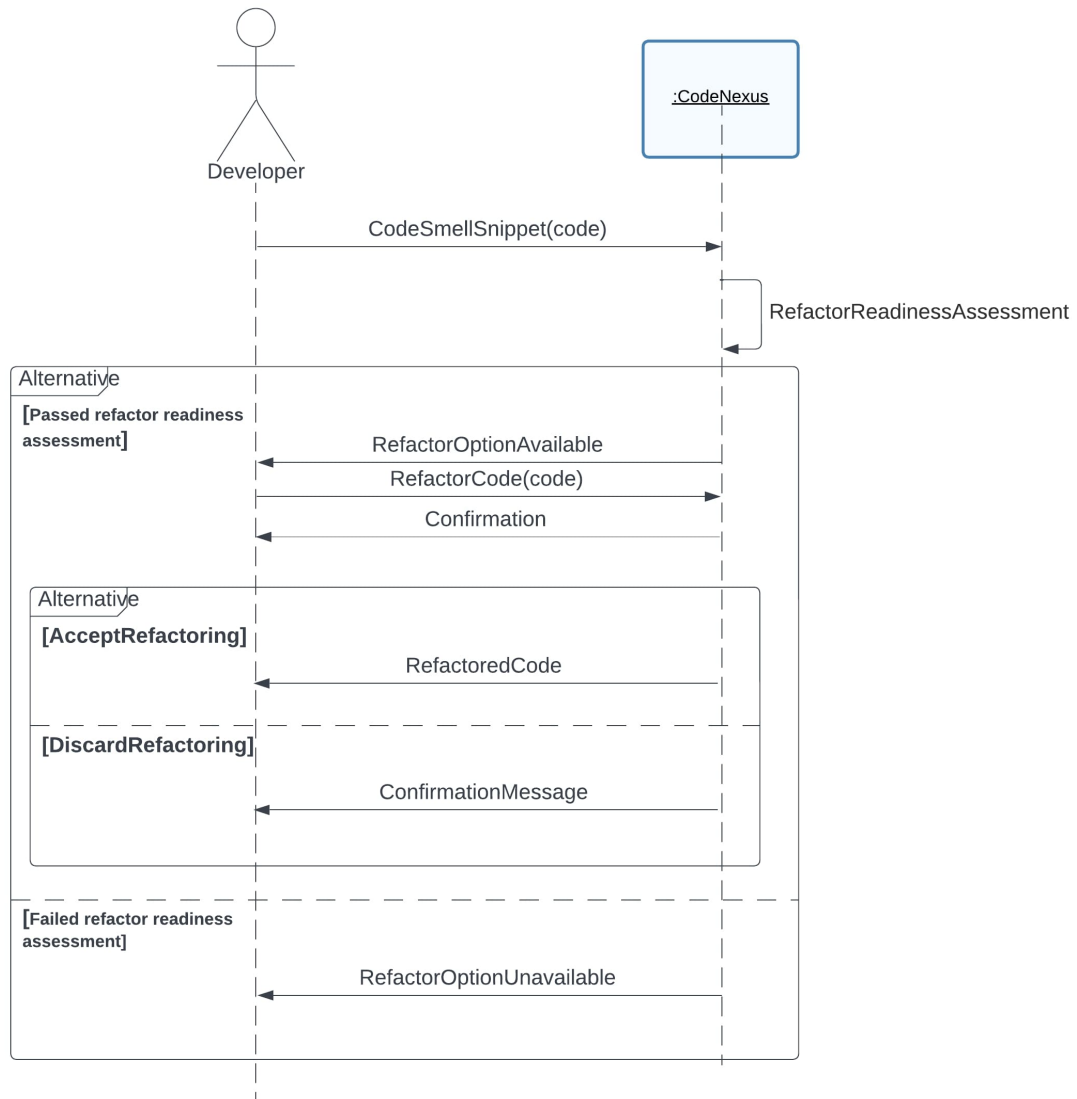


Figure 3.18: UC-05 System Sequence

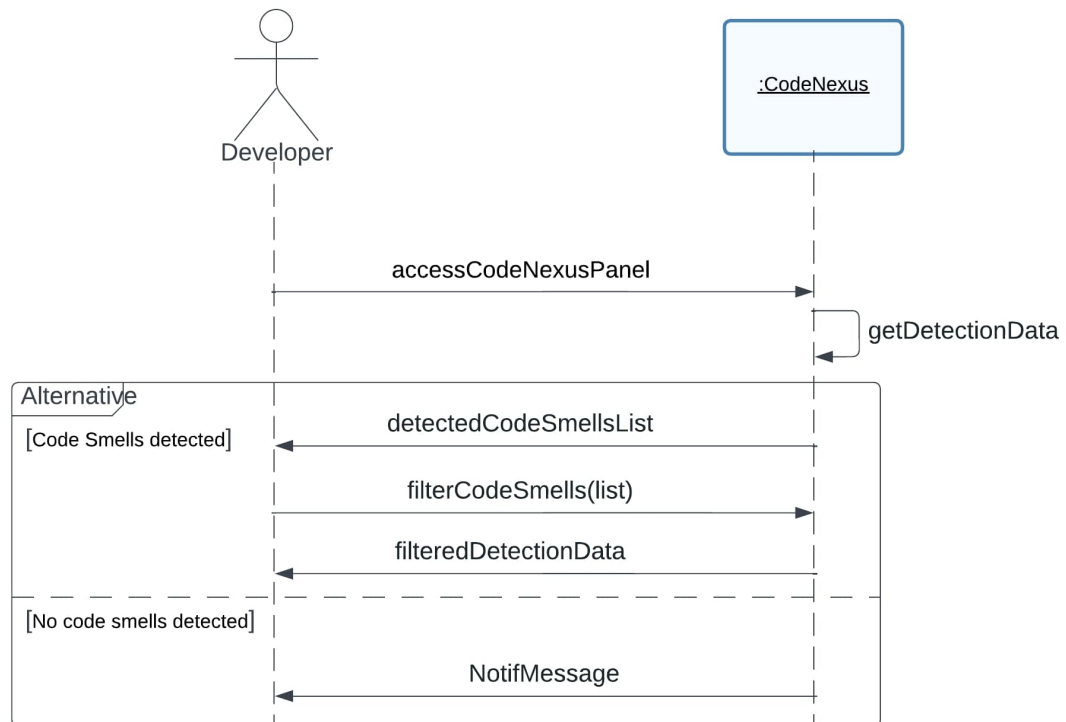


Figure 3.19: UC-06 System Sequence

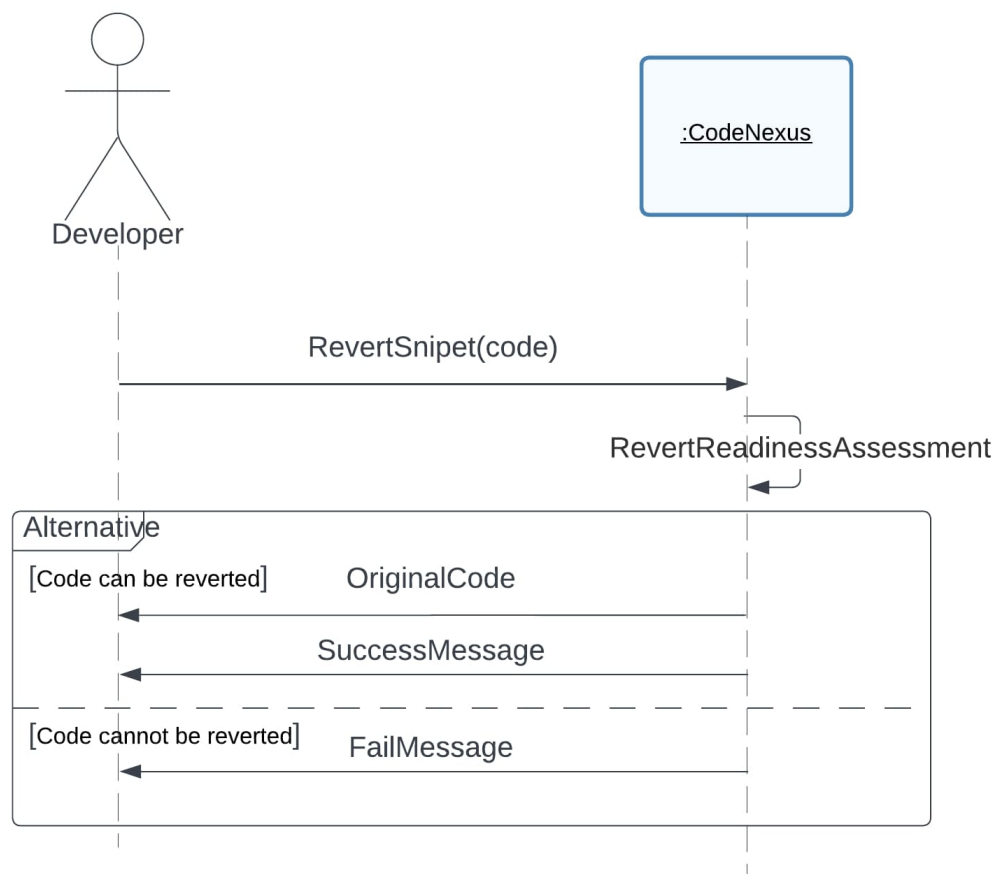


Figure 3.20: UC-07 System Sequence

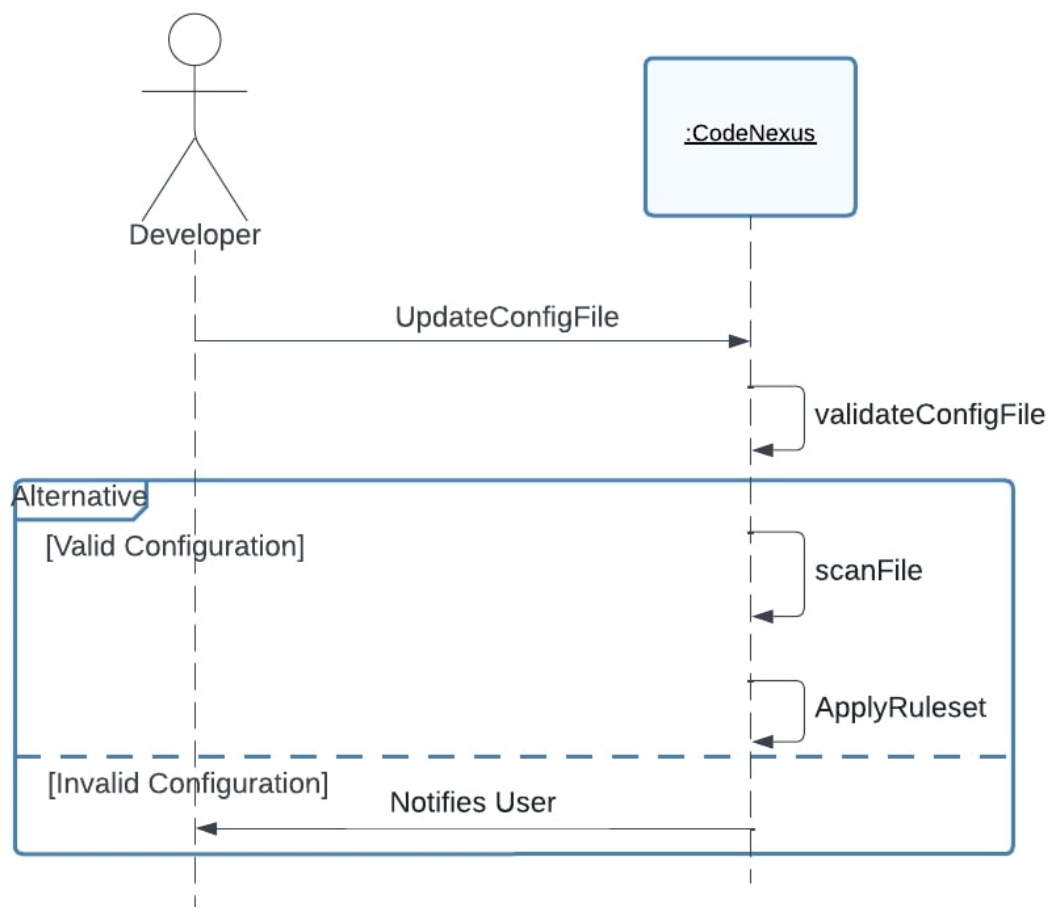


Figure 3.21: UC-08 System Sequence

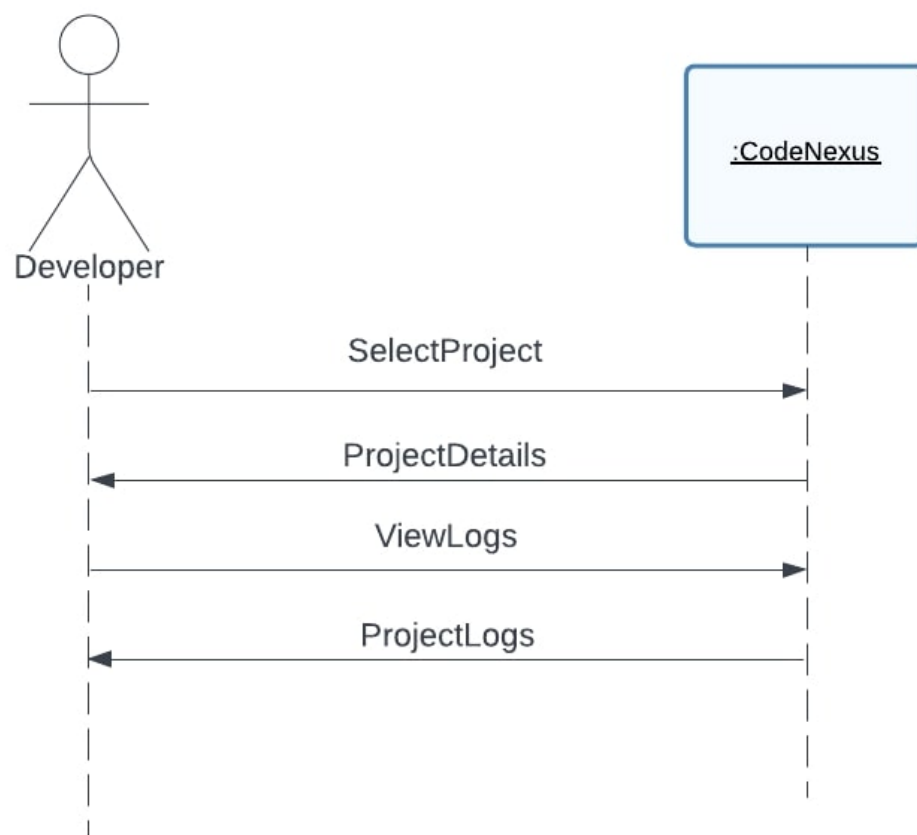


Figure 3.22: UC-09 System Sequence

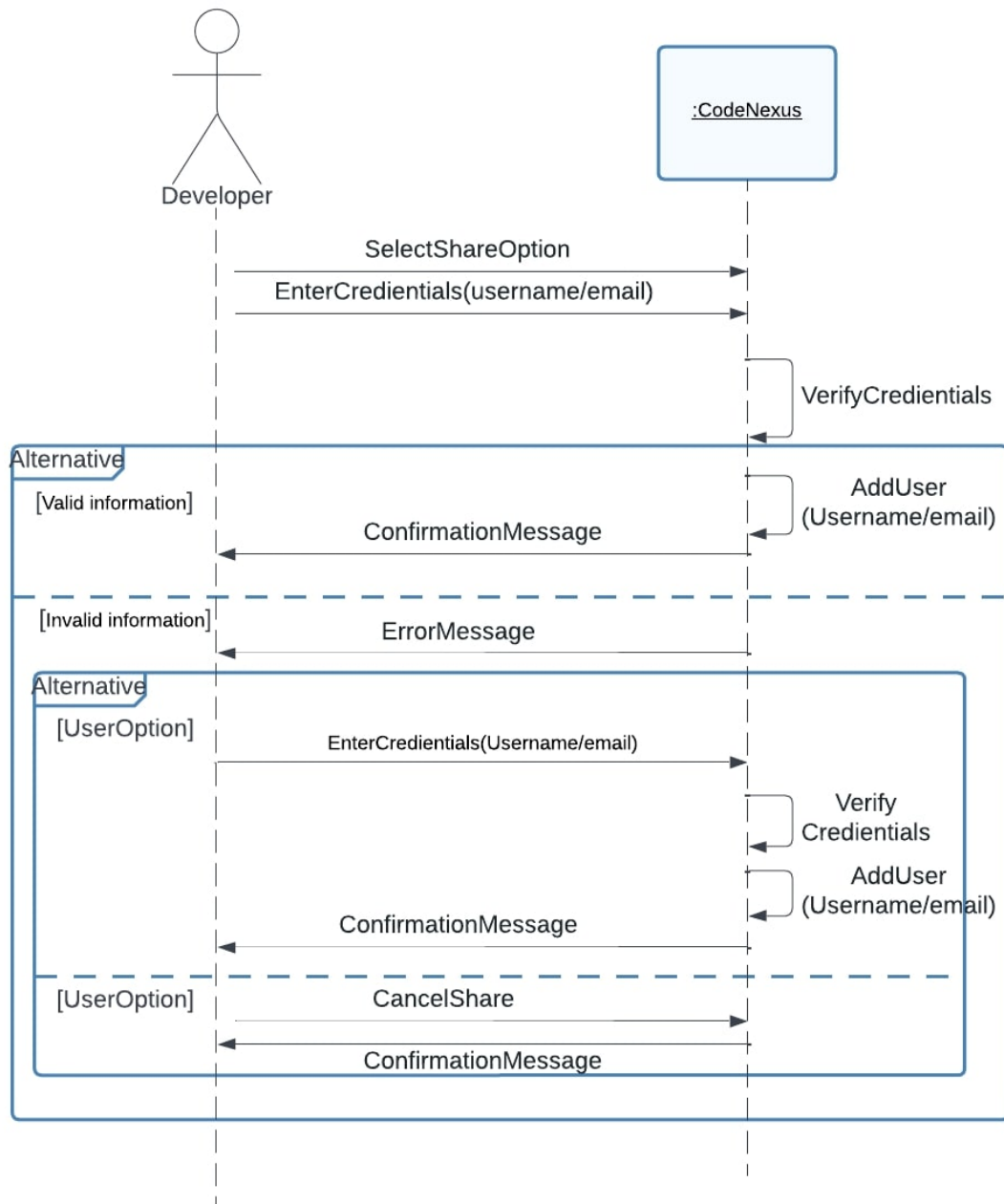


Figure 3.23: UC-10 System Sequence

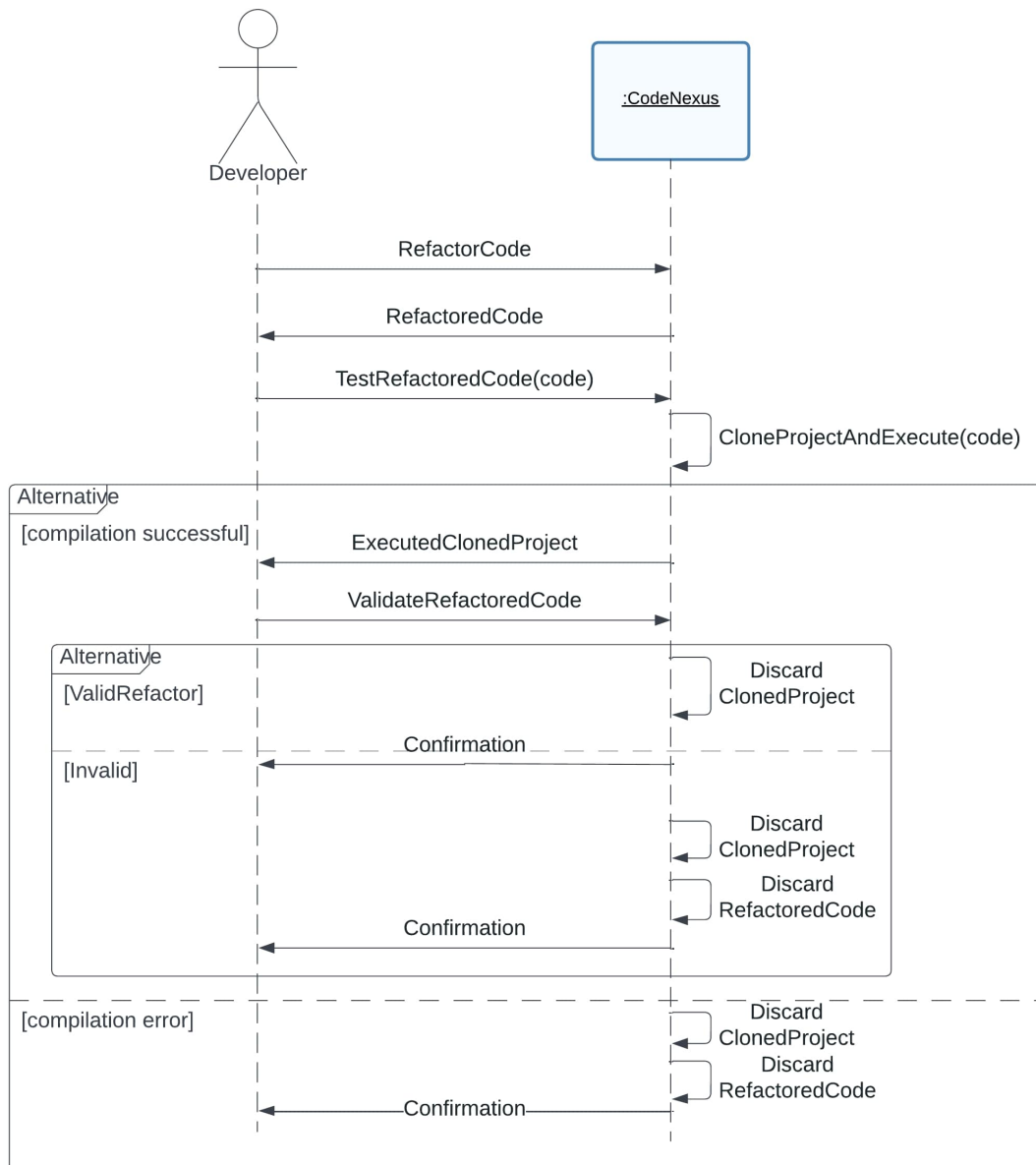


Figure 3.24: UC-11 System Sequence

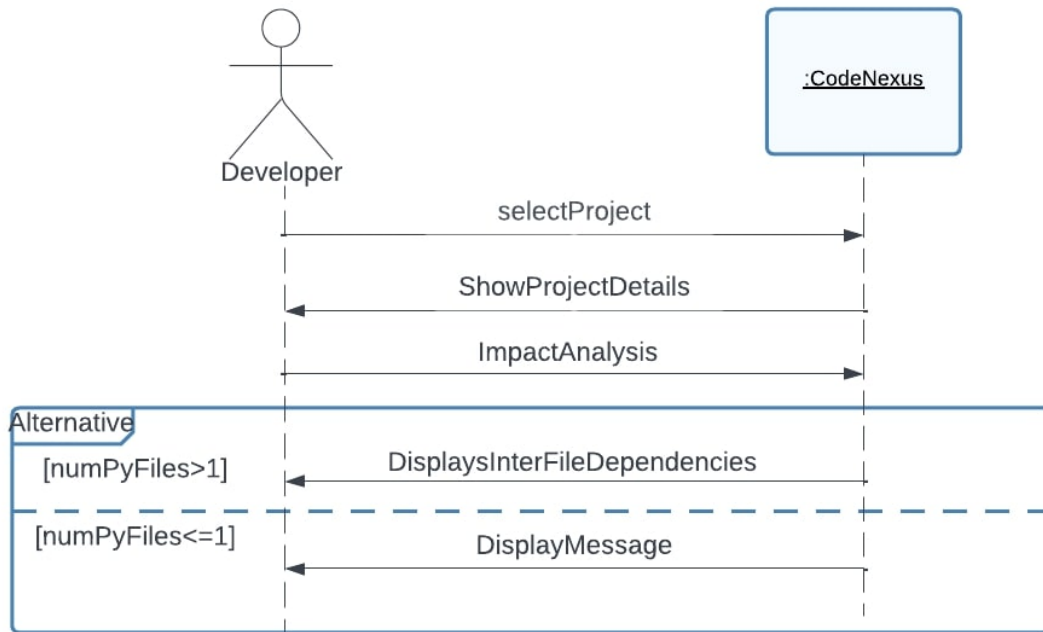


Figure 3.25: UC-12 System Sequence

3.3 Data Design

The JSON schema defines the structure of how data is stored and managed within a VS Code extension for Python projects. Each project has its workspace data, tracking folder structure, file data, inter-file dependencies, and potential code smells. The following key components are detailed in the schema:

- **Project Structure:**

- Each project workspace consists of three primary data objects:

- * **Folder Structure:** This object represents the complete structure of the project directory. The `FolderStructure` includes all files and subfolders, tracking the entire hierarchy of the project's file system.
- * **File Data:** For each Python file (`.py`) in the project, information is maintained:
 - **Code and AST:** The raw code of the file and its parsed Abstract Syntax Tree (AST) are stored for analysis and code analysis.
 - **Functions:** The list of functions defined within the file..

- **Classes:** Details of any classes within the file, capturing variables, methods, and calls.
 - **Global Variables:** Information about all global variables declared in the file.
 - **Imports:** Data on all imports from other files, such as imported modules, classes, or functions, including their names and types. This helps in understanding the dependencies.
- * **Dependency Graph:** A separate object that represents the inter-file dependencies across the project:
 - This is a map of `FileNodes`, where each `FileNode` contains the name of a `.py` file.
 - Each `FileNode` has a set of `DependencyNodes` that represent its relationships with other files within the project.
 - A `DependencyNode` keeps track of a list of `UtilizedEntities`, which are details about functions, classes, or variables that are either:
 - Exported to another file.
 - Imported from another file.
 - This structure allows tracking of how each file interacts with others in terms of sharing and utilizing code elements.
- **Code Smells:**
 - The schema includes an object dedicated to tracking and managing code smells across the project. This data structure supports 10 different types of code smells, each as an object that stores:
 - * **Line Numbers and Description:** The specific parts of the code where the smells were identified, along with a description of the issue.
 - * **Tracking and Analysis:** For each code smell type, attributes are used to track whether the analysis was successful, any errors that occurred, and the specific data related to the smell.
 - This data is important for assessing code quality and identifying potential areas of improvement.
- **Attributes and Data Types:**
 - **FileNode Object:** A separate object that tracks each `.py` file's key attributes, including its name and its code content. It serves as a reference to single files and helps manage their connections to other files in the project through the dependency graph.

- **DependencyNode:** A sub-object that provides the details of a file's dependency, specifying the `UtilizedEntities` shared between files and whether the file imports or exports classes, functions, or variables.

- **CodeSmells Object:** For each code smell detected, details such as `line_numbers`, `unique_violations`, `unused_variables`, and potential conflicts are tracked. The object records success status, errors, and relevant data for detailed analysis.

- **Data Relationships and Integrity:**
 - The `FolderStructure` organizes how files and subfolders are layered within the project.

 - Each file's `File Data` allows for parsing and analyzing the Python code, which is important for features like code navigation, refactoring, and in-depth code quality assessments.

 - The `Dependency Graph` maintains the relationships between different `.py` files, providing a map of inter-file dependencies.

 - The `Code Smells` object is for identifying and tracking issues in the code-base, allowing users to improve code quality and maintainability.

This schema allows the VS Code extension to give a detailed view of the project's structure, code details, dependencies, and code quality, thereby aiding developers in analyzing, and improving their Python projects.

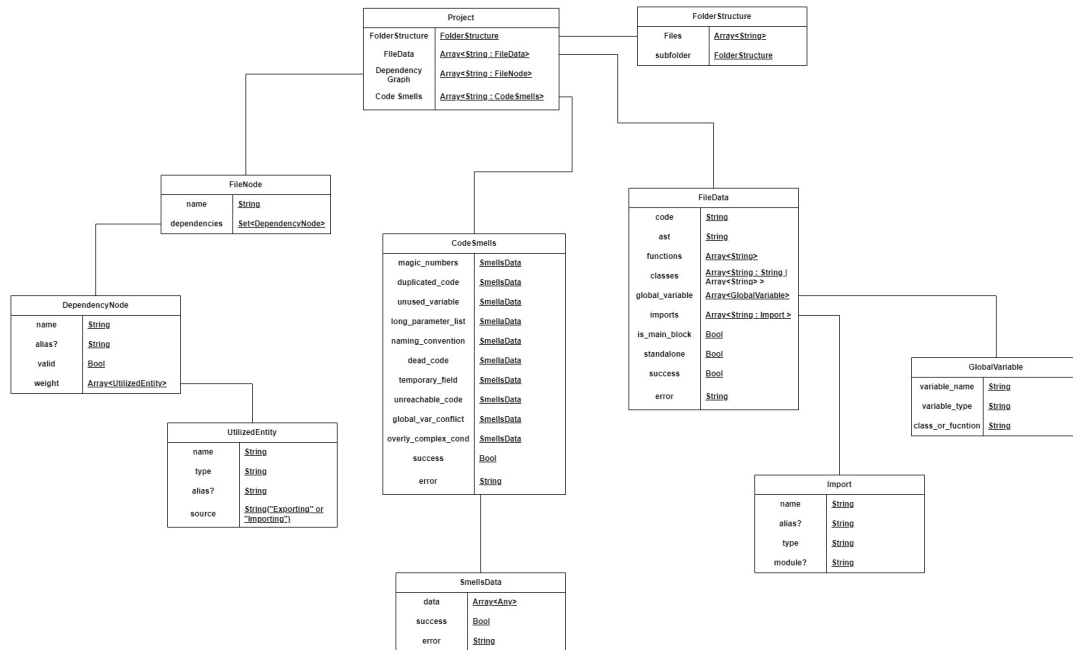


Figure 3.26: JSON Schema for CodeNexus

Glossary

Comments as Code Over-reliance on comments to explain what the code does instead of writing self-explanatory code.. 2

Dead Code Code that is never executed or referenced anywhere in the project.. 2

Duplicated Code Same code segments that are seen in multiple places, making maintenance challenging.. 2

Duplicated Logic Similar Code Segments which perform the same tasks and have the same outputs.. 2

Excessive Use of Flags Using boolean flags in methods to control behavior instead of breaking the logic into separate methods or classes.. 3

Feature Envy A class that seems more interested in the methods of another class than its own.. 2

Global Variables Conflict Local Variables and Global variables with the same names that will add conflicts in code.. 3

God Object A class that knows too much or does too much, violating the Single Responsibility Principle.. 2

Inappropriate Intimacy Classes or modules that are too dependent on each other, making changes difficult.. 2

Inconsistent Naming Variables, methods, or classes with inconsistent naming conventions that make code harder to read.. 3

Long Functions Functions that are excessively long and difficult to read and maintain.. 2

Long Parameter List Methods or functions that take too many parameters, making them hard to understand and maintain.. 2

Magic Numbers Hard-coded values in the code that lack context or explanation.. 3

Middle Man Classes that have only one purpose of being used for interactions with other classes.. 2

Non-Descriptive Method Names Methods that do not clearly indicate their purpose or functionality.. 3

Over-Encapsulation Making everything private when it could be accessed without exposing unnecessary details.. 2

Overly Complex Conditional Statements Long or complicated if statements that are hard to follow.. 2

Switch Statement Abuser Refers to the excessive use of switch statements, especially when they become large and complex. Over-relying on them can lead to hard-to-maintain code and may indicate that better organization using polymorphism or other design patterns is needed.. 2

Temporary Field Variables or fields in a class that serve only one function and remain empty half of the time.. 2

Too Many Nested Loops Deeply nested loops that can lead to performance issues and reduced readability.. 2

Unreachable Code Code that will never be executed. For example, statements after the return statement will never be executed.. 2

Unutilized Variables Variables that are declared but never used in the code.. 3

Variables Naming Notation Refers to inconsistent or unclear naming conventions for variables.. 3

Bibliography

- [1] JetBrains. Python developers survey 2021, 2021. Accessed: 2024-09-08.
- [2] Statista. Most used languages among software developers globally 2024, 2024. Accessed: 2024-09-08.